NETWORK OPERATIONS &

PREDICTIVE INTELLIGENCE SYSTEM

Progressive Lab Guide — Trainer Edition

Integrated build guide and Claude / Claude Code / Cowork usage guide

Telecom Italia Mobile Phone Activity Dataset — Milan Grid

Python • PySpark • Data Engineering • FastAPI • React • Machine Learning • Claude

Revision 2.0

Contents

Contents 2

How to Use This Guide 5

Architecture 7

Dataset Scope & Reference 8

Core Dataset Contract 9

Telecom Precision & Data Handling Rules 12

The Data Scope Ladder 13

The Four Claude Surfaces 14

The Autonomy Model 14

The AI Engineering Record 15

Phase 1 — Core Python: Data Understanding & Reusable Utilities 16

- NP1 — Profile the Telecom Activity Dataset 16
- NP2 — Build the UsageProcessor Class 18
- NP3 — Rule-Based Network Activity Alert Generator 20

Phase 2 — PySpark: Distributed Data Processing 24

- SP1 — Distributed Ingestion 24
- SP2 — Cleaning & Standardization 26
- SP3 — Network Activity Aggregations 27
- SP4 — Geospatial Enrichment Using the Milan Grid 29
- SP5 — Performance & Execution Behaviour 32
- SP6 — Write Processed & Analytics Data 34
- SP7 — Build the Reusable Spark ETL Job 35

Phase 3 — Data Engineering: Productionize the Network Pipeline 38

- DE1 — Design the Telecom Data Architecture 38
- DE2 — Build the Landing-to-Raw Ingestion Flow 40
- DE3 — Orchestrate the Spark Processing Job 41
- DE4 — Batch vs Streaming Decision Workshop 43
- DE5 — Storage Strategy & Data Zones 44
- DE6 — Warehouse Modelling for Network Analytics 46
- DE7 — End-to-End Airflow Orchestration 48
- DE8 — Network Pipeline Reliability Challenge 49

Phase 4 — FastAPI: Network Intelligence Service Layer 52

- API1 — Network Summary Endpoint 52
- API2 — Grid Activity Drill-Down Endpoint 53
- API3 — Hotspot & Alert Endpoint 55
- API4 — Grid Feature Endpoint 56

API5 — Prediction Endpoint — Stub to Real Model ... 58
API6 — Operational Support Endpoints — Pipeline Status & Grid Location ... 59
Phase 5 — React: Lightweight NOC Dashboard ... 62
RE1 — Set Up the NOC Dashboard ... 62
RE2 — Network Overview Page ... 63
RE3 — Grid Explorer ... 64
RE4 — Hotspots, Alerts and the Milan Map ... 66
RE5 — Predictive Risk View ... 68
Phase 6 — Machine Learning: Lightweight Network Activity Intelligence ... 70
ML1 — Frame the Operational ML Problem ... 70
ML2 — Engineer Network Activity Features ... 72
ML3 — Train a Simple Risk Classifier ... 74
ML4 — Add an Anomaly Baseline ... 76
ML5 — Operationalize the Model through FastAPI ... 78
ML6 — Batch Score All Grids ... 79
Phase 7 — Claude: AI-Assisted Network Operations ... 82
C1 — Claude API — Network Insight Generator ... 82
C2 — Tool-Using Claude Network Operations Assistant ... 84
C3 — Long-Context Incident Investigation ... 86
C4 — Introduce Claude Code to the Existing Repository ... 87
C5 — Plan Mode — Add a New NOC Feature Safely ... 89
C6 — Permissions & Security Model ... 90
C7 — Slash Commands & Custom Commands ... 92
C8 — Skills — Package Reusable Network Expertise ... 93
C9 — Subagents & Agent Orchestration ... 94
C10 — Hooks & Event-Driven Workflows ... 96
C11 — Checkpoints & Safe Rollback ... 97
C12 — MCP Server for Network Intelligence ... 99
C13 — Plugins for Team Standardization ... 100
C14 — Claude Agent SDK — Headless NOC Investigation Agent ... 102
C15 — Headless / CI Engineering Review ... 103
C16 — Context, Cost & Usage Optimization ... 105
Final Integrated Capstone — AI-Powered Network Operations Control Room ... 107
Trainer Transition Scripts ... 109
Claude Topic Coverage Map ... 109
Repository Structure ... 111
Appendix A — Lab 0: Environment Setup ... 113
Appendix B — Known Traps, Consolidated ... 114

Appendix C — Indicative Programme Timing 115

Final Trainer Principles 116

Appendix D — What Changed in Revision 2.0 ..... **Error! Bookmark not defined.**

How to Use This Guide

This single document replaces the two documents previously issued. Every lab now carries its build instructions and its Claude usage guidance side by side, so a trainer never has to hold two files open at once and the two can no longer drift apart.

What each lab contains

Section	What it is for
Header strip	Indicative duration, which days of data the lab needs, and the recommended Claude autonomy level.
Scenario and Goal	The business framing. Keep this visible — learners are building one system, not doing isolated exercises.
Student Activities	The build steps.
Concepts	What the trainer should draw out while learners work.
Expected Output	The artifacts that must exist when the lab ends. Later labs consume these by name.
Claude Usage	What the learner must own, what Claude may do, and how much autonomy is appropriate here.
Suggested learner prompt	A ready-to-paste prompt. Learners may adapt it; they may not replace it with “build SP3”.
Learner Validation & Discussion	What the learner must independently verify before accepting Claude’s output.
Acceptance criteria	Objective, checkable pass conditions. The lab is not done until every box is ticked.
Known trap	Failure modes that produce a wrong answer with no error message. Brief learners on these before they start.
Trainer focus	The teaching point, and a question to ask the room.

Principles for the trainer

- Trainer-led progression: each lab reuses outputs from previous labs rather than starting from a fresh toy example.
- Keep the business story visible. Learners are building a Network Operations & Predictive Intelligence System.
- Use the dataset’s real semantics throughout: geographic grid cell, hourly interval, country-code dimension, SMS activity in/out, call activity in/out, internet activity.
- Where simulated metadata or labels are introduced, state clearly that they are training constructs.
- Machine learning is intentionally lightweight. The emphasis is problem framing, features, interpretation and operationalization — not advanced mathematics.
- Claude sits above the data and ML layers. It reasons over curated evidence and tools; it is not a substitute for Spark, SQL, data engineering or ML.
- Generated code is not complete until the learner has validated it against an independent source: a hand calculation, a SQL query, a Spark count, an API response or observed UI behaviour.

Why this edition exists

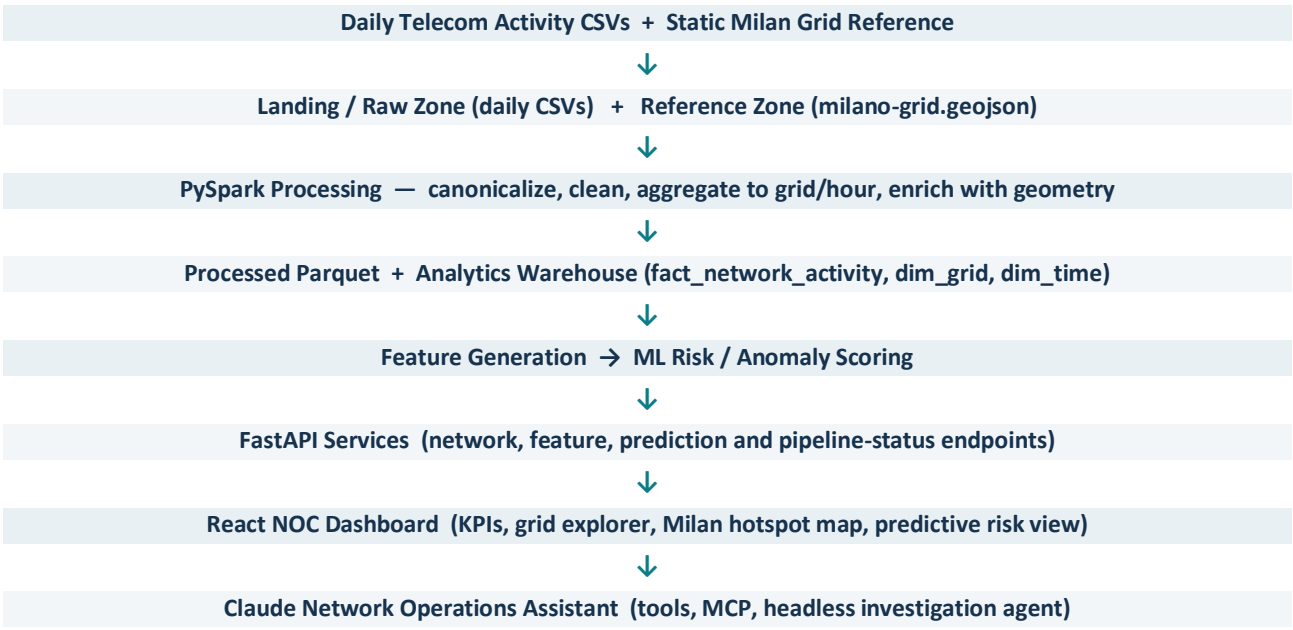
Learners have already completed a separate customer-churn capstone, so they do not need to rebuild every familiar pattern by hand. In this project Claude and Claude Code are used deliberately as an engineering partner: learners specify, plan, delegate, review, test and validate.

The AI is allowed to accelerate implementation. The learner must still own the technical decisions and be able to explain the resulting system.

First capstone: learn how to build the stack. **This project:** learn how to build the stack responsibly with an AI engineering partner.

Architecture

One system, built in layers. Each phase adds a layer and every layer is consumed by the next.



> Responsibility boundary, stated once and enforced everywhere: **Spark and SQL compute. ML scores. Claude reasons over curated evidence.** Claude is never asked to do the work of the analytics stack.

Dataset Scope & Reference

Source dataset: **https://www.kaggle.com/datasets/marcodena/mobile-phone-activity**

Students will be supplied with the correct files directly. They should not download and glob the full Kaggle archive, which contains several other cities and reference sets that will silently corrupt this project's joins.

In scope — the only two inputs this project uses

File	Role	Zone
`sms-call-internet-mi-YYYY-MM-DD.csv`	Hourly grid-level mobile activity. One file per day. This is the only file that flows through the daily ingestion pipeline.	`data/landing/` → `data/raw/`
`milano-grid.geojson`	Static geographic reference for the 10,000-cell Milan grid. Never changes, never versioned by date, never ingested daily.	`data/reference/`

Explicitly not used in this version

If any of these appear in a learner's working folder, remove them. They are listed by name so that nobody quietly adds them "because they were in the download".

- `mi-to-provinces-YYYY-MM-DD.csv` — directional province-to-province call data
- `Italian_provinces.geojson` — province boundaries
- `ISTAT_census_variables_2011.csv` — census/demographic enrichment
- `sms-call-internet-tn-YYYY-MM-DD.csv` and `trentino-grid.geojson` — the Trentino counterpart of this dataset. Cell IDs overlap numerically with Milan but refer to entirely different geography.
- Any social, news or weather extras bundled with the archive.

> ⚠ **KNOWN TRAP — SILENT, NO ERROR MESSAGE**
>
> Always glob the daily files as `sms-call-internet-mi-*.csv`, never `sms-call-internet-*.csv`.
>
> The looser pattern picks up the Trentino files. Cell IDs collide numerically with Milan, so the load succeeds, the row count roughly doubles, and geographic enrichment coverage silently collapses to about half — with no error anywhere. Bake the `-mi-` into the ingestion pattern in DE2 and into the Spark reader in SP1.

Core Dataset Contract

This is the single source of truth for names, grain, time and geography. Every lab, every API and every CLAUDE.md rule refers back to it. Where a lab and this contract disagree, this contract wins.

1. Schema

Raw column (as delivered)	Canonical project name	Notes
`datetime`	`timestamp`	Hourly interval start.
`CellID`	`grid_id`	Geographic grid cell, 1–10000. Not a tower, BTS, customer or subscriber.
`countrycode`	`country_code`	Country-code dimension. Do not assign an unverified meaning to `country_code = 0`.
`smsin`	`sms_in`	SMS activity received. A proportional activity measure, not a message count.
`smsout`	`sms_out`	SMS activity sent.
`callin`	`call_in`	Call activity received.
`callout`	`call_out`	Call activity issued.
`internet`	`internet_activity`	Internet activity. Not megabytes.

2. Grain — the most important rule in the project

Layer	Grain	Meaning
Raw / canonical	`timestamp` + `grid_id` + `country_code`	One hourly activity record for a grid **and one country-code category**. The same grid and hour therefore appears in several rows.
Main analytics	`timestamp` + `grid_id`	Sum the five activity measures across country-code rows so each grid has exactly one record per hourly interval. This is `hourly_grid_summary`, and everything downstream reads it.

Country-code detail is preserved in the raw and canonical layers and may be used for a separate optional analysis. It must not leak into the operational analytics path, the warehouse fact table, the APIs or the dashboard.

3. Time

- The supplied files are **hourly**. Derive `date`, `hour` and `day_of_week` from `timestamp`.
- Each daily file therefore covers 24 hourly intervals.
- Historical note, worth stating so nobody thinks the guide is wrong: the **original** Telecom Italia release (Barlacchi et al., *Scientific Data*, 2015) used **ten-minute** time slots. The version used in this project is the hourly-aggregated republication. Learners who read the source paper will see “ten minutes” — that refers to the original release, not to these files. Verify the cadence in the supplied files in NP1 rather than assuming it.

4. The AS_OF convention — read this before building any API or dashboard

This is a 2013 dataset being presented as a live network operations picture. Without a single shared definition of "now", each layer invents its own and the dashboard, the API and the Claude assistant end up disagreeing with each other. Define it once, in configuration, and have every layer read it.

Term	Definition	Used by
`AS_OF`	The maximum `timestamp` present in the analytics layer. Held as a single configuration value or a warehouse view, never hardcoded in a component.	Everything
"current reporting window", "right now", "the last hour"	The single hourly interval at `AS_OF`.	API1, API3, RE2, C2, Capstone
"recent", "the last 24 hours"	The 24 trailing hourly intervals ending at `AS_OF`, inclusive.	API2, API4, ML2, RE3
"baseline window"	The trailing history ending at `AS_OF` but **excluding** the current interval. Length is set per lab.	NP3, ML2, ML4
`as_of` query parameter	Every read endpoint accepts an optional `as_of`; when absent it defaults to the configured `AS_OF`. This makes demos reproducible and lets a trainer replay any hour.	API1–API6

> Excluding the current interval from the baseline window is not a stylistic choice. If the current hour is inside its own baseline, every deviation is damped by its own contribution and spike detection quietly under-fires. This is the same discipline that prevents leakage in ML2.

5. Geography

Property	Value
Format	GeoJSON `FeatureCollection`
Feature count	10,000 — the Milan grid is 100 × 100 cells
Grid identifier	`properties.cellId`, integer, **1 to 10000**, unique, no gaps
Geometry	`Polygon` for every feature. Five-point closed ring, roughly 235 m square
Coordinate system	EPSG:4326 (WGS 84), longitude then latitude. Milan sits near 9.19 E, 45.46 N
File size	About 3.2 MB — relevant to RE4, where it is served to the browser
Join key	`properties.cellId` → normalize to `grid_id`, then join to activity on `grid_id`

> **⚠ KNOWN TRAP — THE HIGHEST-RISK DEFECT IN THIS PROJECT**
>
> Each feature carries **two** identifiers: a top-level `"id"` that is **0-based** (array position) and `"properties": {"cellId": …}` that is **1-based**. Feature `id: 0` is cell `1`.
>
> Joining on the top-level `id` succeeds, reports approximately 100% coverage and produces an **off-by-one across all 10,000 cells** — every hotspot renders on its neighbour's polygon. There is no error and no warning, and a coverage percentage cannot detect it.
>
> **Always join on `properties.cellId`.** Validate by spot-checking one known cell's centroid against its expected position, not by counting matches. SP4 carries the check.

Minor note: the file carries a named CRS member (`urn:ogc:def:crs:EPSG::4326`). RFC 7946 deprecates this member, so GeoPandas and some JavaScript parsers emit a warning on load. It is harmless — the coordinates are already WGS 84 longitude/latitude.

6. Derived fields

Field	Definition	Introduced in
`total_sms`	`sms_in + sms_out`	NP1
`total_calls`	`call_in + call_out`	NP1
`total_activity`	`total_sms + total_calls + internet_activity`. A **project-defined composite indicator**, not an official Telecom Italia KPI. Frequently dominated by `internet_activity`, so always retain the individual measures alongside it.	NP1
`activity_growth`	Current window versus prior baseline window	ML2
`peak_ratio`	Peak activity ÷ mean activity over the window	ML2
`variability`	Standard deviation, or a coefficient-of-variation style proxy	ML2
`internet_share`	`internet_activity ÷ total_activity`	ML2
`active_hours`	Count of hours in the window with activity above zero	ML2

7. Expected volumes — use these as acceptance checks

Let **D** be the number of daily files loaded.

Check	Expected value
Distinct `timestamp` values in the cleaned data	exactly `D × 24`
Distinct `grid_id` values	≤ 10000, and every value within 1–10000
Rows in `hourly_grid_summary`	≤ `D × 24 × 10000`, and **zero duplicates** on (`grid_id`, `timestamp`)
Rows in raw / canonical layer	Strictly greater than `hourly_grid_summary` — because of the country-code dimension. If they are equal, the country-code aggregation did not happen.
Geographic enrichment coverage	100% of distinct `grid_id` values match a polygon — provided the join used `properties.cellId`

> A grid with no activity in a given hour may be absent from the raw file rather than present with zeros, so the summary row count is an upper bound rather than an exact figure. The **duplicate check is the strict one** and it is the single most valuable test in the project: a duplicate on (`grid_id`, `timestamp`) means the country-code aggregation was missed or partially applied, and every KPI, feature, model score and API response downstream is then inflated. Run it at SP3 and again after every pipeline change.

Telecom Precision & Data Handling Rules

These rules exist because this dataset is unusually easy to overclaim with. Enforce them in review, in the CLAUDE.md written at C4, and in the wording used on the dashboard.

Language

- The SMS, call and internet fields are **activity measures**, not literal message counts, call counts or megabytes. Say "SMS activity", "call activity", "internet activity".
- `total_activity` is a **project-defined composite indicator**. Never present it as an official telecom KPI.
- This dataset measures communication activity, **not radio or network capacity**. High activity does not mean congestion. There is no throughput, latency, packet-loss or utilization data anywhere in this project.
- Use: high-activity risk, hotspot, unusual activity, anomaly, operational attention signal. If congestion is mentioned at all, label it explicitly as a simulated or proxy training target.
- `grid_id` is a geographic cell. Never a tower, BTS, customer, subscriber or account.
- Do not import customer-retention or churn concepts into this track. That was the previous capstone.

Data handling

- Preserve the raw files **unchanged**. Every transformation writes elsewhere.
- Profile blank activity fields **before** transforming them. Do not drop an otherwise valid row merely because one activity measure is blank — blanks are common in this dataset and are informative.
- The curated layer may apply a documented **null-to-zero rule** to the activity-measure columns before aggregation. Missing `grid_id` or `timestamp` remains a data-quality failure and is quarantined, never defaulted.
- Record how many nulls the curated rule handled. That count is a data-quality metric, and it is evidence the Claude assistant will use later.

> **TRAINER NOTE — CHECKS THAT ARE SUPPOSED TO RETURN NOTHING**
>
> NP1 and SP2 ask learners to check for **negative activity values** and **exact duplicates**. On genuine supplied files both checks return **zero** — activity values are non-negative proportional measures and the raw grain is naturally unique.
>
> Tell learners this in advance. Otherwise they assume they wrote the check incorrectly and spend the session debugging correct code. These checks earn their place at DE2 and DE8, where the trainer deliberately injects faulty files and they fire as designed.

The Data Scope Ladder

Learners are not asked to tackle the full data complexity on day one. Scope widens deliberately, and each widening is motivated by a capability the previous phase has just introduced. Announce this ladder at the start of the programme so learners understand that the scope change *is* the lesson.

Phase	Data in play	Why it widens here
Phase 1 — Core Python NP1–NP3	**One daily file.** 24 hourly intervals, one day.	Build and validate the processing logic where a learner can still read the data by eye and check a number by hand. Correctness first, scale later.
Phase 2 — PySpark SP1–SP7	**Multiple daily files** — the folder of supplied days.	The same logic, now distributed. This is the point of Spark: the business rules do not change, the execution model does. Multi-file loading is introduced in SP1.
Phase 3 — Data Engineering DE1–DE8	**Files arriving one at a time**, plus the accumulated history already processed.	Airflow turns the Spark job into a repeatable pipeline where each new day's file is detected, validated, routed and processed automatically. Reliability and reruns matter here, not volume.
Phases 4–7 — API, React, ML, Claude	**The accumulated processed history** in the warehouse and analytics layer.	Serving, modelling, visualization and the assistant all read what the pipeline has already built. They never read raw CSVs.

Minimum days required per phase

The exact number of days supplied is a trainer decision. These are the minimums below which specific labs stop working — not preferences.

Labs	Minimum days	What breaks below the minimum
NP1, NP2	1	Nothing. These are designed for a single file.
NP3	1	Nothing — **provided** the baseline is the within-day baseline specified in NP3. A per-grid, per-hour-of-day baseline is impossible from one day, because each bucket would contain exactly one observation and every deviation would be zero.
SP1–SP7	3 or more	Multi-file loading, `input_file_name` traceability, partitioning by date and day-of-week comparisons all become degenerate with one or two files.
DE1–DE8	3 or more, delivered one at a time	Idempotency, duplicate-ingestion handling and safe reruns need more than one arrival to demonstrate.
ML2, ML3, ML4	**14 recommended, 10 absolute minimum**	Below this the chronological train/test split has too little history on either side, per-grid baselines are noisy, and precision/recall becomes unstable enough to be misleading.
Capstone	Same as ML	The assistant needs enough history to answer "has this happened before".

> If the programme runs with fewer than ten days for the ML phase, say so explicitly to the room and treat the evaluation numbers as illustrative. That is a perfectly acceptable training outcome. Presenting an unstable precision figure as if it were meaningful is not.

The Four Claude Surfaces

Learners will use four different ways of working with Claude across this programme, and confusing them is the most common cause of wasted effort. Introduce this table in the first session and refer back to it whenever a lab specifies a "primary AI mode".

Surface	What it is	Use it in this project for	Do not use it for
Claude (chat)	Conversation. No access to the repository or the data.	Explaining concepts, critiquing a design the learner has already written, challenging an ML problem statement, rehearsing an argument before committing to it.	Anything that requires reading actual files. It will guess the schema.
Claude Code	An agent resident in the repository, with the terminal and the file system.	Inspecting the real CSV and GeoJSON, scaffolding modules, refactoring across files, running tests, debugging pipeline failures. This is the workhorse for Phases 1–6.	Producing polished documents for humans. It writes code, not deliverables.
Claude Cowork	An agent that works on files and produces finished artifacts — documents, spreadsheets, dashboards, reports.	The written deliverables this project generates: the NP1 profiling summary, the DE1 architecture one-pager, the DE4 decision matrix, the DE8 failure-handling matrix, the ML3 evaluation report, the SP5 performance notes, and the final capstone investigation brief.	Editing the repository. Keep source code in Claude Code.
Claude API	Claude embedded inside the product being built.	Phase 7 — the Network Operations Assistant, tool use, MCP, and the headless investigation agent. Claude stops being the thing that helps build the platform and becomes part of it.	Day-to-day development assistance.

> The distinction learners most often miss: **Claude Code owns the repository, Cowork owns the deliverables.** A learner who asks Claude Code to produce a client-ready architecture document gets a Markdown file in the repo; a learner who asks Cowork gets a document they can hand over. Several labs in this guide produce written artifacts for a named audience — those are Cowork labs, and each one is flagged in its Claude Usage table.

The Autonomy Model

Every lab specifies one of four modes. The mode is a trainer instruction, not a suggestion — it is what keeps a learner from delegating away the exact decision the lab exists to teach.

Mode	Learner owns	Claude does
LEARN	The core concept and the first attempt	Explains, questions and reviews. Does not write the solution.
PAIR	The approach and the key decisions	Scaffolds or completes parts of the implementation after the approach is agreed.
DELEGATE	The acceptance criteria and the validation	Implements a substantial portion of the task.
REVIEW	The final decision and any correction	Challenges, diagnoses, compares and proposes improvements. Does not decide.

Autonomy is deliberately **low** where the dataset can mislead — data grain, ML framing, chronological validation, architecture. It is deliberately **high** where the pattern was already learned in the previous capstone — FastAPI boilerplate, React components, storage syntax.

Before the formal Claude module (Phase 7), learners may use ordinary Claude and Claude Code capabilities: repository inspection, planning, coding, debugging, refactoring, testing. They should **not** depend on Skills, Subagents, Hooks, MCP or the Agent SDK before those topics are taught. A plan-first prompt is fine; formal Plan Mode arrives at C5.

The AI Engineering Record

Required for every Claude-assisted lab. Four lines. This is the primary assessment device in the programme and it is what separates a learner who is engineering from one who is transcribing.

1. What I asked Claude to do.
2. What Claude changed, generated or recommended.
3. What I validated independently — data, SQL result, test, API response, UI behaviour, metric or Spark output.
4. **What I changed or rejected from Claude's suggestion, and why.**

The fourth line is the one that matters. A record where every entry reads "accepted as generated" across forty labs is itself the finding — surface it in review rather than letting it pass.

Trainer guardrails for AI-assisted labs

- Do not accept one-shot prompts such as "build SP3" or "complete DE7". Require a clear task, an acceptance criterion and a validation step.
- For data-grain, ML framing, chronological validation, architecture and Spark-performance decisions, the learner must explain the choice — before or after Claude assists, but they must explain it.
- Generated code is not complete until the learner runs tests or validates output against an independent source.
- Claude should inspect existing repository patterns before creating new abstractions. Watch for duplicate service, feature or transformation logic.
- Use the real data semantics in prompts so Claude cannot drift back to customer churn, customer IDs, towers or fabricated telecom fields.
- Keep the excluded files out of the build unless a trainer deliberately adds a future enrichment extension.

Phase 1 — Core Python: Data Understanding & Reusable Utilities

Purpose	Understand the event and time-series nature of the telecom activity data, and build reusable Python utilities that later evolve into Spark and pipeline components.
Storyline	Raw telecom activity → profile → clean → derive KPIs → produce rule-based operational alerts.
Data scope	**One daily file.** Everything in this phase is designed so a learner can still check a number by hand.
Indicative duration	
Autonomy note	Starts LEARN, rises to PAIR/DELEGATE. Claude may not write the first profiling code.

NP1 — Profile the Telecom Activity Dataset

■ Data: one daily CSV ◆Autonomy: Low–Medium (LEARN / PAIR)

Business / Training Scenario

The NOC data team receives a raw network activity extract. Before building any processing logic, learners must understand what one row represents and identify the data-quality risks that will follow the data all the way to the dashboard.

Goal

Understand the source structure, quality and time/grid characteristics before any transformation.

Student Activities

1. Load one `sms-call-internet-mi-YYYY-MM-DD.csv` file with pandas and inspect shape, raw columns and dtypes.
2. Document the raw-to-canonical mapping and standardize `datetime`, `CellID`, `countrycode`, `smsin`, `smsout`, `callin`, `callout` and `internet` into the canonical project schema.
3. Parse the `timestamp` field and **verify the hourly cadence in the supplied file** — confirm there are exactly 24 distinct timestamps and that consecutive intervals are one hour apart. Then derive `date`, `hour` and `day_of_week`.
4. Check for missing `grid_id`, missing `timestamp`, blank activity measures, exact duplicates and negative activity values. Keep the raw file unchanged and document the curated-layer null policy separately.
5. Inspect how many country-code rows exist for the same grid and hour. Confirm the raw grain is `timestamp` + `grid_id` + `country_code`.
6. Create `total_sms`, `total_calls` and `total_activity` as clearly labelled **derived** activity measures.
7. Compute the profiling facts: number of unique grids, time range, cadence, country-code categories, busiest hourly window, busiest grid, and null counts per column.
8. Write a five-bullet data profiling summary addressed to the Network Analytics Team.

Concepts

- `read_csv`, dtypes, `isnull`, `duplicated`, `value_counts`
- datetime conversion, cadence validation and time-derived fields
- the difference between raw country-code-level activity records and grid/hour analytics aggregations
- grid-based activity is not customer-level CRM data

Expected Output

- Profiling report — the written five-bullet summary for a named audience
- Canonicalized DataFrame using the project schema
- Initial derived activity measures

Claude Usage for This Lab

Learner owns	Understand what a raw row means, identify the raw grain, discover the real columns, and decide which quality checks are necessary.
Primary AI mode	Claude Code for repository and data inspection; Claude for explanation.
Meaningful AI activity	Claude should inspect the **actual supplied CSV** rather than assume a generic telecom schema. It may prepare the profiling checklist and explain findings, but the learner makes the first profiling decisions and writes at least part of the pandas exploration themselves.
Recommended autonomy	Low–Medium (LEARN / PAIR)
Where Cowork fits	The five-bullet profiling summary is a written deliverable for a named audience — a good first Cowork task once the learner has the numbers.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Explore one sms-call-internet-mi-YYYY-MM-DD.csv file in this repository
without modifying it.

First report:
- the exact raw column names and likely data types
- whether multiple rows can exist for the same datetime + CellID because of
  countrycode, with a concrete example
- the actual time cadence, evidenced by the distinct timestamps in the file

Then propose a profiling plan covering:
- raw-to-canonical column mapping
- missing CellID / datetime
- blank activity fields
- exact duplicates
- negative activity values
- country-code categories
- hourly cadence
- number of unique grids
- derived total_sms, total_calls and total_activity

Do NOT implement the full pandas solution yet.

Explain what one raw row represents, and clearly distinguish the raw grain
from the later grid/hour analytics grain. Treat the activity values as
proportional activity measures, not counts or megabytes.
```

Learner Validation & Discussion

- Learner manually verifies the raw column names against the file rather than accepting Claude's list.
- Learner can explain why Grid 4821 at 00:00 may appear in several rows with different `countrycode` values.
- Learner checks that Claude has not interpreted activity values as literal counts or MB anywhere in its output.

- Learner writes or completes the initial pandas profiling code before asking Claude to review it.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Exactly 24 distinct `timestamp` values in the file, one hour apart.

☐Every `grid_id` falls within 1–10000.

☐A worked example is documented showing one `grid_id` + `timestamp` appearing in more than one row, with the differing `country_code` values listed.

☐Null counts per column are recorded, and the blank-activity count is non-zero and explained rather than silently removed.

☐The written summary uses "activity" language throughout and contains no reference to message counts, megabytes, towers or customers.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

The negative-value check and the exact-duplicate check will both return **zero** on a genuine supplied file. This is the correct result, not a bug in the learner's code. Say so before they start — otherwise the room spends forty minutes debugging code that works. Both checks earn their place at DE2 and DE8, where faulty files are deliberately injected.

TRAINER FOCUS

Ask the room: *"What does one row represent?"* The answer is: one hourly activity record for a grid **and one country-code category**. Then ask why several rows can exist for the same grid and hour, and why the downstream NOC dataset needs a grid/hour aggregation. Do not allow customer IDs, plans, churn, tower IDs or physical BTS assumptions into this track.

Trainer check: Ask the learner "what is the grain of the raw file?" A correct answer names all three parts: timestamp, grid and country code.

↳ **Connects to:** NP2; SP1

NP2 — Build the UsageProcessor Class

■ **Data: one daily CSV** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

The analytics team wants the same cleaning and KPI logic applied repeatedly to different daily files, without copying code between notebooks.

Goal

Turn one-off pandas operations into reusable, testable processing logic.

Student Activities

1. Create a `UsageProcessor` class that accepts a file path or a DataFrame.
2. Implement `load_data()`, `clean_data()`, `derive_time_features()`, `aggregate_to_grid_time()`, `derive_activity_features()`, `compute_kpis()` and `export_summary()`.
3. Add defensive checks for missing required columns, missing `grid_id` or `timestamp`, negative activity values, and the documented curated-layer null handling rule.
4. Add Python logging for load counts, dropped rows and output paths. Use logging, not `print`.

5. Generate one-record-per-grid/hour analytics data, plus daily and grid-level summary tables. Preserve `country_code` only in the raw/canonical layer or in a separate optional analysis.
6. Write one small unit-style validation for each major method.

Concepts

- OOP, methods, state and separation of concerns
- exceptions and defensive coding
- logging instead of print for production visibility
- reusability and explicit data-grain control that transfers directly into the Spark and DE pipeline

Expected Output

- `usage_processor.py`
- `daily_summary.csv`
- `grid_summary.csv`
- structured execution log

Claude Usage for This Lab

Learner owns	Class responsibilities, cleaning policy, the data-grain transition, and method boundaries.
Primary AI mode	Claude Code — design review, then partial implementation.
Meaningful AI activity	Claude may design and implement substantial boilerplate, because learners already know Python. The learning is in the separation between raw loading, cleaning, grid/hour aggregation and KPI generation.
Recommended autonomy	Medium–High (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review the NP1 implementation and propose a clean UsageProcessor design.

The class should include:
  load_data()
  clean_data()
  derive_time_features()
  aggregate_to_grid_time()
  derive_activity_features()
  compute_kpis()
  export_summary()

Requirements:
- preserve the raw input unchanged
- apply the documented curated-layer rule for blank activity measures,
  and report how many nulls it handled
- reject missing grid_id / timestamp and negative activity values
- aggregate country-code rows to one grid/hour record BEFORE any
  operational KPI is computed
- keep country_code in the raw/canonical layer only, not in the main
  grid/hour analytics output
- add logging and clear exceptions
```

```
Show the method responsibilities first. Do NOT implement until I approve
the design.
```

Learner Validation & Discussion

- Learner explains why `aggregate_to_grid_time()` is a separate step rather than folded into `clean_data()`.
- Learner reviews whether Claude applied the null policy **only** in the curated layer, and not to the loaded raw frame.
- Learner checks that KPIs are calculated on the grid/hour grain, not on the country-code grain.
- Learner reviews the generated tests, exceptions and logging rather than accepting them blindly.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Running `aggregate_to_grid_time()` produces zero duplicates on (`grid_id`, `timestamp`).

☐The output of `aggregate_to_grid_time()` has strictly fewer rows than its input.

☐`country_code` is absent from the grid/hour analytics output.

☐The log reports input rows, rows rejected, nulls handled by the curated rule, and output rows — as four separate numbers.

☐Each of the seven methods has at least one validation, and all pass.

TRAINER FOCUS

The lesson is not sophisticated OOP. It is that the same transformation logic should not be copied into every script — and that the grain transition deserves its own named method so it can be tested and, later, ported to Spark unchanged.

Trainer check: Ask: "Why must `compute_kpis()` not also load and clean the file?" The answer should address separation of concerns and testability.

↳ **Connects to:** NP3; SP2; SP7

NP3 — Rule-Based Network Activity Alert Generator

■ **Data: one daily CSV — see the baseline definition below** ◆**Autonomy: Medium–High (PAIR / DELEGATE)**

Business / Training Scenario

Before any predictive model exists, the NOC still needs basic rules that surface unusual activity patterns for investigation. This becomes the first serving artifact in the system, and it is later consumed by the API, the dashboard and the Claude assistant.

Goal

Create a transparent, pre-ML operational alert layer — and understand exactly what it cannot see.

Student Activities

1. Start from the grid/hour analytics table produced by `aggregate_to_grid_time()` in NP2.
2. Build the **within-day baseline**: for each `grid_id`, compute the median `total_activity` across that day's 24 hourly intervals, **excluding the hour currently being evaluated**. Use the median rather than the mean so a single extreme hour does not raise its own baseline.
3. Apply an activity floor. Grids with very low daily totals produce meaningless ratios and will otherwise dominate the alert list. Choose the floor from the data and document the choice.

4. Define three training rules against that baseline: `HIGH_ACTIVITY` (current hour materially above the grid's own within-day baseline), `ACTIVITY_SPIKE` (sharp rise against the immediately preceding hour) and `ACTIVITY_DROP` (current hour materially below the grid's own baseline).
5. For each grid and hour, compare current activity against the baseline and apply the three rules.
6. Create alert records containing `grid_id`, `timestamp`, `alert_type`, `current_activity`, `baseline_activity` and `reason`.
7. Write alerts to CSV or JSON and print a short operational summary: alerts by type, top ten grids by alert count, and the proportion of all grid/hours that alerted.
8. Discuss false positives, and write down explicitly what this baseline **cannot** distinguish.

Concepts

- functions, conditions and dictionary/JSON output
- baseline thinking — and the difference between a baseline and a threshold
- transparent business rules versus learned ML patterns
- operational alert design: an alert is a request to investigate, not a diagnosis

Expected Output

- `network_alerts.csv` or `network_alerts.json`
- rule-based alert summary
- a written statement of the within-day baseline's limitations
- the first serving artifact — later consumed by API3, RE4 and the Claude assistant

Claude Usage for This Lab

Learner owns	Approve the baseline definition and every threshold. Understand false positives and the limits of activity-only data.
Primary AI mode	Claude Code as implementation partner; Claude as threshold critic.
Meaningful AI activity	Claude may implement most of the reusable alert module **after** the learner approves the rules. It must operate on the one-record-per-grid/hour analytics table, never on raw country-code rows.
Recommended autonomy	Medium–High (PAIR / DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Using the grid/hour analytics table created after country-code aggregation,
propose a simple transparent alert design for:
  HIGH_ACTIVITY
  ACTIVITY_SPIKE
  ACTIVITY_DROP

IMPORTANT — we currently have ONE day of data. The baseline must therefore
be a WITHIN-DAY baseline: for each grid, the median total_activity across
that day's 24 hourly intervals, excluding the hour being evaluated.
Do NOT propose a per-grid, per-hour-of-day baseline. With one day of data
each of those buckets holds exactly one observation, so every deviation
would be zero.

For every rule, show:
```

```
- required fields
- threshold logic and the activity floor you would apply to small grids
- reason text
- likely false positives
- what this within-day baseline structurally CANNOT detect
- what additional data would be required before anyone could call this
  "network congestion"

Do NOT implement until I approve the thresholds. After approval, implement
the reusable alert module producing grid_id, timestamp, alert_type,
current_activity, baseline_activity and reason.
```

Learner Validation & Discussion

- Learner approves or modifies **every** threshold, including the activity floor, and can justify each number from the data.
- Learner compares at least one alert against the underlying hourly rows by hand.
- Learner can explain why the output is an operational attention signal, not a fault diagnosis.
- Learner writes down the specific blind spot: a within-day baseline cannot tell “this grid is normally quiet at 03:00” from “this grid has dropped”, because it has no concept of what a normal 03:00 looks like.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ The baseline for a given grid/hour demonstrably excludes that hour’s own value — verifiable by hand on one grid.

☐ Alert volume is operationally plausible. If a large share of all grid/hours are alerting, the thresholds are wrong and must be revised before the lab is signed off.

☐ Every alert record carries a human-readable `reason` naming the rule, the current value and the baseline value.

☐ The alert file loads cleanly and its `grid_id` values all exist in the Milan grid.

☐ A written limitation statement exists and names the hour-of-day blind spot.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

A per-grid, **per-hour-of-day** baseline cannot be built from a single day. Each bucket would contain exactly one observation — the current value — so the baseline would equal the current value, every deviation would be zero, and all three rules would return nothing. Learners who reach for the obvious `groupby(["grid_id","hour"])` will produce an empty alert file and assume they have a bug.

This lab uses the within-day baseline instead. The hour-of-day baseline arrives at **ML4**, once Phase 2 and Phase 3 have accumulated history — and the fact that it *needs* that history is precisely the point to make here.

TRAINER FOCUS

This lab replaces customer-plan enrichment and creates the conceptual bridge the whole programme runs on: **rules first, ML later, Claude explanation last.** The within-day baseline’s weakness is not a compromise to apologise for — it is the motivation for ML4. Ask the room what data they would need to fix it, and let them arrive at “more days” themselves.

Trainer check: Ask: “Grid 4821 shows 40% of its daily activity in one hour. Is that an alert?” The answer depends on whether that is normal for 4821 — which this baseline cannot tell you. That is the lesson.

↳ **Connects to:** ML1; ML4; API3; Claude C1

Phase 2 — PySpark: Distributed Data Processing

Purpose	Rebuild the network processing flow using Spark so learners understand distributed ingestion, transformation, aggregation, joins, storage and execution behaviour.
Storyline	Same telecom problem, same KPIs, different engine: pandas concepts now scale to multi-file distributed processing.
Data scope	**Multiple daily files.** Scope widens here because Spark is what makes it possible — the business logic does not change, the execution model does.
Indicative duration	Approximately 12–13 hours
Autonomy note	Mostly PAIR/DELEGATE. Schema, grain and join decisions stay with the learner.

SP1 — Distributed Ingestion

■ Data: all supplied daily files ◆Autonomy: Medium–High (PAIR / DELEGATE)

Business / Training Scenario

Network activity arrives as many daily files. Learners must load the whole collection as one distributed DataFrame instead of looping over files in pandas.

Goal

Read multiple telecom activity files with Spark and enforce a reliable schema.

Student Activities

1. Create a `SparkSession`.
2. Read the daily files from a folder using the pattern `sms-call-internet-mi-*.csv`. Use the `-mi-` in the glob — see the trap below.
3. Compare `inferSchema` against a trainer-provided manual `StructType`, and explain the cost of each.
4. Count rows, source files, unique grids, country-code categories and distinct hourly intervals.
5. Add an `input_file_name` column for traceability.
6. Inspect the partition count and explain why file layout affects Spark execution.

Concepts

- SparkSession
- manual schema versus inference
- distributed file loading
- lazy evaluation
- traceability

Expected Output

- `raw_network_df`
- validated schema
- file-level row count report

Claude Usage for This Lab

Learner owns	Spark schema choices, multi-file loading and source-file traceability.
Primary AI mode	Claude Code — Spark scaffolding.
Meaningful AI activity	Claude may scaffold the ingestion module, but it must use the **actual** raw CSV schema and the real daily file pattern rather than a generic telecom schema.
Recommended autonomy	Medium–High (PAIR / DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect the actual daily sms-call-internet-mi-YYYY-MM-DD.csv files and the
canonical schema used in the Python labs.

Create a PySpark ingestion module that:
- starts a SparkSession
- reads multiple daily CSV files using the glob sms-call-internet-mi-*.csv
  (the -mi- is required; a looser pattern would pick up files from a
  different city whose cell IDs collide numerically with Milan)
- uses a manual StructType rather than inferSchema
- preserves the raw country-code-level grain
- adds the source filename for traceability
- reports row count, file count, unique grids, country-code categories
  and distinct hourly intervals

Explain why you selected each Spark data type.
Do not invent tower, customer or subscriber fields.
```

Learner Validation & Discussion

- Learner compares the manual schema against `inferSchema` and can state what inference costs in extra passes over the data.
- Learner intentionally changes one field type and observes the resulting failure or silent coercion.
- Learner confirms Spark has **not** prematurely collapsed country-code rows — the row count should still reflect the raw grain.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Distinct `timestamp` count equals `D × 24`, where D is the number of files loaded.

☐File count reported by the job equals the number of files actually in the folder.

☐All `grid_id` values fall within 1–10000.

☐Row count is strictly greater than the eventual `hourly_grid_summary` count, confirming the country-code grain is intact.

☐Every row carries a populated `input_file_name`.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

Glob on `sms-call-internet-mi-*.csv`, never `sms-call-internet-*.csv`. The looser pattern picks up the Trentino files if they are present in the folder. Their cell IDs overlap numerically with Milan, so the read succeeds, the row count roughly doubles and geographic enrichment coverage silently halves at SP4 — with no error at any point.

> **TRAINER FOCUS**
>
> Use the actual raw schema first, then normalize to the canonical project schema. The raw grain is `timestamp` + `grid_id` + `country_code`. Treat `grid_id` as a geographic cell and the supplied files as hourly data.

↳ **Connects to:** SP2

SP2 — Cleaning & Standardization

■ **Data: all supplied daily files** ◆**Autonomy: High coding, learner owns validation (DELEGATE + REVIEW)**

Business / Training Scenario

Spark has loaded all the files, but downstream teams need correct timestamps, numeric measures and clean identifiers before anything can be trusted.

Goal

Produce a trusted distributed DataFrame for network analytics.

Student Activities

1. Rename the raw columns to the canonical names from the Core Dataset Contract.
2. Cast the activity measures to numeric types and `timestamp` to a usable datetime, then verify the expected hourly cadence still holds across all files.
3. Quarantine rows with missing `grid_id` or `timestamp`, or with negative activity values. Profile blank activity measures and apply the documented curated-layer null-to-zero rule **only after** raw preservation.
4. Create `total_sms`, `total_calls` and the project-defined `total_activity` indicator, while retaining the original SMS, call and internet measures.
5. Derive `date`, `hour` and `day_of_week`.
6. Compare record counts before and after cleaning, capture rejected-row counts, and report how many activity nulls the curated-layer rule handled.

Concepts

- `withColumn`
- `cast`
- `filter`
- `dropna`
- `when` / `otherwise`
- column expressions

Expected Output

- `clean_network_df`
- rejected-record summary
- null-handling report

Claude Usage for This Lab

Learner owns	Validate that the Spark implementation is semantically equivalent to the pandas rules already agreed.
Primary AI mode	Claude Code — translation plus test generation.

Meaningful AI activity	Claude can translate the already-understood pandas cleaning rules into Spark and generate comparison tests between the two implementations.
Recommended autonomy	High coding, learner owns validation (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review the approved pandas cleaning rules from NP1/NP2 and implement the
equivalent PySpark transformations.

Requirements:
- rename the actual raw columns to the canonical project names
- cast datetime and activity fields correctly
- verify the hourly cadence holds across all loaded files
- quarantine missing grid_id / timestamp and negative activity values
- profile blank activity fields, then apply the documented null-to-zero
  rule ONLY in the curated layer
- derive date, hour and day_of_week
- retain the individual SMS, call and internet measures
- create total_sms, total_calls and the project-defined total_activity

Also create a small comparison test that runs the pandas UsageProcessor and
the Spark path over the SAME single day and asserts the outputs match.
```

Learner Validation & Discussion

- Learner compares pandas and Spark row counts and sample values for the same day — they must match exactly.
- Learner verifies the null-handling counts, and can say how many nulls were converted and in which columns.
- Learner explains why `total_activity` must remain labelled as a project-defined composite indicator.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ The pandas-versus-Spark comparison test passes on at least one full day.

☐ Rejected-row count and null-handled count are reported as separate numbers, not merged.

☐ Distinct `timestamp` count is still `D × 24` after cleaning.

☐ The individual `sms_in`, `sms_out`, `call_in`, `call_out` and `internet_activity` columns still exist alongside the composites.

↳ **Connects to:** SP3

SP3 — Network Activity Aggregations

■ **Data: all supplied daily files** ◆**Autonomy: High (DELEGATE) — but grain validation is learner-owned**

Business / Training Scenario

NOC and operations managers do not inspect raw events. They need time and grid summaries and hotspot rankings.

Goal

Generate operational KPIs using distributed groupings — and make the grain transition explicit and testable.

Student Activities

1. Collapse the country-code-level records to one row per `timestamp` + `grid_id` by summing `sms_in`, `sms_out`, `call_in`, `call_out` and `internet_activity` across country-code categories.
2. From the consolidated grid/hour DataFrame compute total SMS activity, total call activity, internet activity, `total_activity`, and daily activity per grid.
3. Identify the top ten high-activity grids for selected windows.
4. Compute the peak activity hour.
5. Compute internet share of total activity.
6. Create `hourly_grid_summary` as the canonical downstream analytics DataFrame: exactly one record per `grid_id` + hourly `timestamp`.

Concepts

- `groupBy`
- `agg`
- `sum`
- `avg`
- `orderBy`
- window function concepts
- grain as a testable property

Expected Output

- `hourly_grid_summary`
- `daily_traffic_summary`
- hotspot ranking

Claude Usage for This Lab

Learner owns	The transformation from country-code grain to grid/hour analytics grain. This decision is not delegable.
Primary AI mode	Claude Code — implementation partner.
Meaningful AI activity	Claude can implement the aggregation functions, but learners must explicitly approve the grain and the aggregation rule before any code is written.
Recommended autonomy	High (DELEGATE) — but grain validation is learner-owned

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Using clean_network_df, implement reusable Spark aggregations.

First, collapse the raw/canonical country-code rows to ONE row per
timestamp + grid_id by summing:
  sms_in, sms_out, call_in, call_out, internet_activity
across country_code.

Then compute:
- total SMS activity
- total call activity
- internet activity
```

```
- total_activity
- daily activity by grid
- top 10 high-activity grids
- peak activity hour
- internet share

Create hourly_grid_summary as the canonical downstream analytics table,
with exactly one record per grid_id + hourly timestamp.

Add an assertion that hourly_grid_summary has ZERO duplicates on
(grid_id, timestamp), and fail the job if it does not.

After implementation, explain which Spark operations are transformations,
which trigger execution, and where shuffles are likely.
```

Learner Validation & Discussion

- Learner inspects one specific grid and hour before and after country-code consolidation, and confirms the sums by hand.
- Learner predicts at least one aggregation result before executing it.
- Learner verifies that `hourly_grid_summary` has exactly one record per grid and hourly timestamp.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ **Zero duplicates on (`grid_id`, `timestamp`) in `hourly_grid_summary`.** This assertion must be in the code, not just run once interactively.

☐ `hourly_grid_summary` row count is strictly less than `clean_network_df` row count.

☐ Row count is at most `D × 24 × 10000`.

☐ Hand-checking one grid/hour reproduces the aggregated value exactly.

☐ `country_code` does not appear in `hourly_grid_summary`.

TRAINER FOCUS

The country-code aggregation is **mandatory** for the main NOC path. Call high values “high-activity grids” or “hotspots”; do not claim congestion unless capacity or utilization data is introduced, which it never is in this project.

Trainer check: This is the single most important validation point in the programme. A wrong grain here contaminates every later KPI, model score, API response and dashboard number — and it does so silently, by inflating everything proportionally. Do not let the room move on until the duplicate assertion is in the code and passing.

↳ **Connects to:** SP4; DE6; RE2

SP4 — Geospatial Enrichment Using the Milan Grid

■ **Data: all supplied daily files + `milano-grid.geojson`** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

The telecom activity data identifies location only through `grid_id`. By itself, a value such as 4821 is just an identifier. The project also contains `milano-grid.geojson`, which supplies the geographic polygon for each Milan grid cell. Learners connect activity to geometry so that network activity can be displayed geographically in the NOC dashboard.

Goal

Joins at scale — enrich telecom activity with the actual geographic boundaries of Milan grid cells, and validate the join properly.

Student Activities

1. Load `milano-grid.geojson` and inspect its structure: the top-level type, where the grid identifier is stored, and the geometry type.
2. Identify the common key between the telecom activity dataset and the GeoJSON. In the GeoJSON it is `properties.cellId`; in the project schema it is `grid_id`.
3. Normalize the identifier: flatten `features[]` into a lookup of `grid_id` + `geometry`, mapping `properties.cellId` → `grid_id`.
4. Inspect the size of the grid lookup relative to the activity DataFrame — 10,000 rows against many millions.
5. Perform a left join between the processed network activity data and the Milan grid lookup on `grid_id`.
6. Validate the join by checking the count of distinct activity grids before the join, the count after, the number of grids with missing geometry, and the percentage successfully enriched.
7. Validate the join **geographically as well as numerically** — see the acceptance criteria. A coverage percentage alone is not sufficient evidence that the join is correct.
8. Compare the Spark execution plan for a standard join against a broadcast join, and explain why the grid lookup is a broadcast candidate.
9. Create an enriched dataset containing `timestamp`, `grid_id`, `sms_in`, `sms_out`, `call_in`, `call_out`, `internet_activity`, `total_activity` and `geometry`.
10. Identify the top high-activity grids for a selected window and retain their geometry for later visualization.
11. Optionally derive the centroid of each grid polygon, for simpler map visualizations and API responses.

Concepts

- inner join versus left join, and what each one hides
- geospatial enrichment as a data-engineering problem, not a GIS problem
- GeoJSON structure: FeatureCollection, features, properties, geometry
- Polygon geometry and coordinate order (longitude, latitude)
- joining business data with geographic reference data
- broadcast joins and when the size asymmetry justifies one
- join quality checks and enrichment coverage
- geographic identifiers versus physical network assets

Expected Output

- `grid_activity_geo_df` — the geographically enriched network activity dataset
- grid enrichment coverage report
- list of unmatched `grid_id` values, if any
- top high-activity grids with geometry retained
- processed output suitable for later map visualization

Claude Usage for This Lab

Learner owns	How the real Milan grid reference connects `grid_id` to geography; the join strategy; and the validation that the join is correct rather than merely complete.
Primary AI mode	Claude Code — GeoJSON inspection plus join implementation.

Meaningful AI activity	Claude should work with the **actual** `milano-grid.geojson`, not synthetic zone metadata. It may help flatten the GeoJSON and implement the join after the learner approves the design.
Recommended autonomy	Medium–High (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect data/reference/milano-grid.geojson and the current
hourly_grid_summary.

Report:
- the GeoJSON top-level structure
- EVERY identifier present on a feature, including any top-level "id"
  as well as anything inside "properties"
- the geometry type
- how the identifier should be normalized to grid_id

Then propose a Spark-friendly grid lookup containing grid_id + geometry.
Explain whether you would use a left or inner join, and whether broadcast
is appropriate given the relative sizes.

Do NOT implement until I approve the plan.

After approval:
- prepare and flatten the GeoJSON lookup
- join it to the processed activity on grid_id
- report unmatched grid IDs and enrichment coverage
- ALSO print the centroid of three named grid cells so I can check them
  against their expected position on a map — coverage percentage alone
  will not prove the join is correct
- retain top high-activity grid geometry for visualization
- optionally derive centroids

Important: treat the GeoJSON as static reference data, not a daily input.
Do not invent residential, commercial or transport zone labels.
```

Learner Validation & Discussion

- Learner chooses left versus inner join and justifies the choice in terms of what each would hide.
- Learner compares the standard and broadcast join plans and can point to the difference in the physical plan.
- Learner verifies enrichment coverage **and** performs the geographic spot-check described in the acceptance criteria.
- Learner explains why `grid_id` is a geographic cell identifier, not a tower or BTS.
- Learner confirms that full geometry will not be duplicated into every warehouse fact row at DE6.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The lookup is built from `properties.cellId`, and the code contains a comment or assertion saying so.

☐Enrichment coverage is 100% of distinct `grid_id` values.

☐**Geographic spot-check passes:** the centroid of a named grid cell is printed and lands in the expected part of Milan, and the centroids of `grid_id` 1 and `grid_id` 2 are adjacent rather than identical or far apart.

☐The unmatched-grid list is produced and is empty.

☐Row count after the left join equals the row count before it — a left join must not multiply rows, and if it does, the lookup contains duplicate keys.

☐The enriched output uses `grid_id` and `timestamp`, matching the Core Dataset Contract.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

Every feature in `milano-grid.geojson` carries **two** identifiers: a top-level `"id"` that is **0-based**, and `"properties": {"cellId": …}` that is **1-based**. Feature `id: 0` is cell `1`.

A learner — or Claude — reaching for `feature["id"]` produces a join that succeeds, reports **100% coverage**, and is wrong for all 10,000 cells. Every hotspot renders on its neighbour's polygon. Nothing errors. The coverage check cannot detect it, because every key still matches something.

This is why the acceptance criteria demand a **geographic** spot-check and not just a coverage percentage. Brief learners on this before they open the file, and ask them at the end which identifier they joined on.

TRAINER FOCUS

Make it explicit that `grid_id` represents a geographic grid cell, not a customer, tower or BTS. `milano-grid.geojson` is a FeatureCollection with `cellId` in feature properties and Polygon geometry; learners must flatten `features[]` into a `grid_id` + `geometry` lookup before joining. The GeoJSON tells us **where** activity occurred. It does not tell us whether the area is residential, commercial or transport — the objective is data enrichment, not a GIS deep dive. For the learning exercise, join geometry to activity and compare a standard join with a broadcast join. For the production-style warehouse at DE6, keep geometry as a static dimension and store only `grid_id` plus an optional centroid in the fact table.

Trainer check: The learner should be able to say: "The GeoJSON tells us where the grid is; it does not tell us what type of locality it is." Then ask which identifier they joined on, and why.

↳ **Connects to:** SP5; SP6; DE6; RE4; Claude C2

SP5 — Performance & Execution Behaviour

■ **Data: all supplied daily files** ◆**Autonomy: Low implementation / High explanation (REVIEW)**

Business / Training Scenario

The same transformations can run very differently depending on partitions, reuse and selected columns.

Goal

Develop a practical Spark performance mindset grounded in evidence rather than folklore.

Student Activities

1. Run `explain()` on a hotspot aggregation and read the physical plan.
2. Cache a reused cleaned DataFrame and compare repeated action timings.
3. Repartition by date or another suitable key and observe the partition counts.
4. Demonstrate column pruning by selecting only the required fields before an aggregation.
5. Broadcast the static grid lookup from SP4 and compare the plan against the standard join.
6. Discuss why over-partitioning a small local dataset makes performance worse.
7. Document three performance observations with the evidence that supports each.

Concepts

- `cache` / `persist`
- `repartition`
- `explain`
- lazy evaluation
- column pruning
- broadcast joins

Expected Output

- execution comparison notes
- optimized transformation version

Claude Usage for This Lab

Learner owns	Interpret Spark execution behaviour and choose optimizations based on evidence rather than on a list of suggestions.
Primary AI mode	Claude as diagnostic explainer; Claude Code for minimal changes only after approval.
Meaningful AI activity	Claude should diagnose and explain. It should **not** automatically "optimize everything" — an unrequested optimization pass is the failure mode to watch for here.
Recommended autonomy	Low implementation / High explanation (REVIEW)
Where Cowork fits	The three documented performance observations are a written artifact — a natural Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review the Spark transformations developed so far, including the grid/hour
aggregation and the geospatial enrichment.

Identify possible opportunities for:
- caching / persisting
- repartitioning
- column pruning
- broadcast joining the static grid lookup

Do NOT change any code yet.

For each suggestion, tell me what evidence I should look for in explain()
or in repeated action timings BEFORE applying it.

Also identify any optimization that is likely to be unnecessary or actively
harmful on a training dataset of this size running locally.
```

Learner Validation & Discussion

- Learner runs `explain()` and uses the evidence before accepting any change.
- Learner chooses which optimizations to apply and which to reject, and records the reasoning.
- Learner can explain why over-partitioning hurts small local workloads.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ Three performance observations are documented, each with the specific evidence that supports it.

☐ At least one of Claude's suggestions is **rejected**, with a stated reason.

☐ A before-and-after timing is captured for the cached DataFrame across repeated actions.

TRAINER FOCUS

Avoid turning this into a benchmark contest on laptops. The goal is understanding cause and effect. A learner who applies every suggestion without evidence has missed the lab entirely, even if the job runs faster.

↳ **Connects to:** SP6

SP6 — Write Processed & Analytics Data

■ **Data: all supplied daily files** ◆**Autonomy: High (DELEGATE)**

Business / Training Scenario

The processing job should leave behind reusable cleaned and aggregated datasets, not just display results in a notebook.

Goal

Persist Spark outputs in formats appropriate for downstream analytics.

Student Activities

1. Write the clean activity data as Parquet.
2. Partition the cleaned output by `date`.
3. Write `hourly_grid_summary` as Parquet at one record per grid and hour. Keep full Polygon geometry in the static grid reference rather than duplicating it into every analytics record.
4. Write a small dashboard summary as CSV for easy inspection, and retain `milano-grid.geojson` separately under `data/reference/` for map rendering.
5. Read the Parquet output back and validate schema and counts.
6. Compare file sizes and explain the benefits of columnar storage.

Concepts

- Parquet
- `partitionBy`
- overwrite versus append
- round-trip validation
- why geometry does not belong in a fact-shaped file

Expected Output

- `data/processed/activity/`
- `data/analytics/hourly_grid_summary/`
- `dashboard_summary.csv`

Claude Usage for This Lab

Learner owns	Why different data products use different storage patterns.

Primary AI mode	Claude Code — implementation partner.
Meaningful AI activity	Claude may implement most of the storage code, because the learning objective is the storage **decision** and its validation, not the write syntax.
Recommended autonomy	High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add the output layer to the Spark project.

Requirements:
- cleaned activity -> Parquet
- partition the cleaned output by date
- hourly_grid_summary -> Parquet, one row per grid/hour
- dashboard summary -> CSV
- keep milano-grid.geojson separately in data/reference/
- do NOT duplicate full Polygon geometry into every hourly analytics row
- add a round-trip validation that reads the Parquet back and checks
  schema, row count, and zero duplicates on (grid_id, timestamp)

Explain when append versus overwrite is appropriate for each output.
```

Learner Validation & Discussion

- Learner explains why Parquet is appropriate for the processed and analytics layers but not for the raw zone.
- Learner explains why the static GeoJSON stays in reference data.
- Learner checks the partition layout on disk and the round-trip counts.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ Round-trip read reproduces the exact row count and schema.

☐ The duplicate assertion on (`grid_id`, `timestamp`) still passes after the round trip.

☐ The processed directory is partitioned by date, verifiable from the folder names on disk.

☐ No geometry column exists in `hourly_grid_summary`.

☐ A file-size comparison between the CSV and Parquet forms is recorded.

↳ **Connects to:** SP7; DE5; DE6

SP7 — Build the Reusable Spark ETL Job

■ **Data: all supplied daily files** ◆**Autonomy: High (DELEGATE + REVIEW)**

Business / Training Scenario

The NOC platform needs one executable Spark job that can run against every valid batch, rather than a folder of notebooks.

Goal

Create the bridge from Spark exercises to a production-style processing component.

Student Activities

1. Create `spark/telecom_pipeline.py` with `read_raw()`, `clean()`, `aggregate()`, `enrich()`, `write_outputs()` and `main()`.
2. Accept input, output and reference paths as arguments or configuration — nothing hardcoded.
3. Add logging for input rows, rejected rows, nulls handled, output rows, start and end time, and final status.
4. Fail cleanly and loudly when no input files are present.
5. Run the full job against the training folder.
6. Document the job contract: expected inputs, outputs and failure conditions.

Concepts

- modular ETL
- configuration over hardcoding
- structured logging
- fail-fast behaviour
- the job contract as a document

Expected Output

- `spark/telecom_pipeline.py`
- job execution log
- documented input/output contract

Claude Usage for This Lab

Learner owns	The final Spark job contract, and how the existing transformations are reused rather than rewritten.
Primary AI mode	Claude Code — high-autonomy refactoring.
Meaningful AI activity	An ideal delegation task: Claude can inspect all prior Spark work and refactor it into a production-style job. The learner's job is to confirm nothing was silently reimplemented.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review the Spark work created in SP1-SP6 and propose a refactoring plan for
spark/telecom_pipeline.py.

Requirements:
- reuse existing logic; no duplicate transformations
- functions for read_raw(), clean(), aggregate(), enrich(), write_outputs()
  and main()
- country-code aggregation must occur BEFORE any grid/hour operational
  analytics
- the static milano-grid.geojson must be read from the reference area
- configurable input / output / reference paths
- structured logging of input rows, rejected rows, nulls handled and
  output rows
- clear failure when no daily activity files are present
- documented inputs, outputs and failure conditions
```

```
Show the plan and the files/functions you intend to reuse BEFORE making any
changes. After I approve, execute it and show the git diff summary.
```

Learner Validation & Discussion

- Learner reviews the diff rather than the description of the diff.
- Learner must explain at least three material changes Claude made and why each was necessary.
- Learner runs the full job end to end and verifies the expected outputs appear.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ The job runs to completion from a single command with no manual steps.

☐ Running it against an empty folder fails with a clear message and a non-zero exit code.

☐ The log contains input rows, rejected rows, nulls handled, output rows and status as distinct fields.

☐ No transformation logic exists in two places — the learner can point to the single definition of each rule.

☐ The documented contract matches what the job actually does.

↳ **Connects to:** DE1; DE2; DE3

Phase 3 — Data Engineering: Productionize the Network Pipeline

Purpose	Turn the Spark processing component into a governed data pipeline with ingestion zones, orchestration, storage design, warehouse modelling and reliability controls.
Storyline	Landing → validate → raw → Spark → processed → analytics warehouse → serve → monitor.
Data scope	**Files arriving one at a time**, plus the accumulated history already processed. Reliability matters here, not volume — hold back at least two daily files so arrivals can be simulated.
Indicative duration	Approximately 14–15 hours
Autonomy note	Alternates deliberately. Architecture and batch-versus-streaming stay REVIEW; implementation labs are DELEGATE.

DE1 — Design the Telecom Data Architecture

■ **Data: design lab — no data processing** ◆**Autonomy: Advisor only (REVIEW)**

Business / Training Scenario

The operator wants a repeatable network intelligence pipeline. Teams must define each layer, its responsibility, its tool and its data contract before automating anything.

Goal

Make learners design the full data flow before automating individual tasks.

Student Activities

1. Draw the architecture from the daily activity CSV source plus the static `milano-grid.geojson` reference through to the React and Claude consumers.
2. Map the landing, raw, reference, processed and analytics layers to file formats and tools. Treat `milano-grid.geojson` as static reference data, not a daily ingestion file.
3. Identify the quality gates that must pass before raw acceptance and before analytics publication.
4. Define the analytics outputs: `hourly_grid_summary`, `daily_grid_summary`, hotspots, and the alert and risk tables.
5. Define the responsibilities of Spark, SQL, Airflow, FastAPI, React, ML and Claude — one sentence each, with no overlap.
6. Produce a one-page design document with explicit assumptions and non-goals.

Concepts

- pipeline layers
- data contracts
- separation of responsibilities
- batch architecture
- non-goals as a design tool

Expected Output

- architecture diagram
- tool responsibility mapping
- quality gate list

- one-page design with assumptions and non-goals

Claude Usage for This Lab

Learner owns	Create the first architecture proposal **before** asking any AI to critique it.
Primary AI mode	Claude as architecture reviewer, not author.
Meaningful AI activity	Claude should challenge the learner's design, especially the distinction between daily source data and static reference data. It should not produce the initial design.
Recommended autonomy	Advisor only (REVIEW)
Where Cowork fits	The one-page design with assumptions and non-goals is a document for a named audience — build it in Cowork once the team has agreed the content.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review our proposed Network Intelligence architecture and CHALLENGE it
rather than agreeing with it.

Check whether we have properly separated:
- daily CSV landing / raw ingestion
- static reference data: milano-grid.geojson
- Spark processing
- processed Parquet
- analytics warehouse
- Airflow orchestration
- FastAPI
- React
- ML
- the Claude assistant (later)

Look for missing contracts, duplicated responsibilities, and components
doing work they should not do.

Also tell me what evidence sources an operations assistant would need
later that our current design does not produce.

Do NOT redesign it from scratch. Critique our design and propose only
changes you can justify.
```

Learner Validation & Discussion

- Learner accepts or rejects each major suggestion explicitly, rather than merging them all.
- Learner can explain why the GeoJSON belongs in a reference zone rather than the daily landing zone.
- Learner can identify which layer calculates data, which orchestrates it, and which later explains it.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ Every component in the diagram has exactly one stated responsibility, and no responsibility appears twice.

☐ The reference zone is drawn separately from the landing/raw flow.

☐ A non-goals list exists and includes "we do not have capacity, throughput or utilization data".

☐ The design names where pipeline health will be recorded — this is what API6 later exposes.

> **TRAINER FOCUS**
>
> Explicitly ask: *"Which component calculates data? Which component orchestrates it? Which component explains it?"* Most confusion later in the programme traces back to a team that never answered this cleanly.

↳ **Connects to:** DE2; API6

DE2 — Build the Landing-to-Raw Ingestion Flow

■ **Data: one valid daily file + one deliberately invalid file** ◆**Autonomy: High (DELEGATE)**

Business / Training Scenario

Daily `sms-call-internet-mi-YYYY-MM-DD.csv` activity files arrive in a landing zone. Valid files should move to raw; invalid files should be quarantined and logged. `milano-grid.geojson` is static reference data and stays under `data/reference/` rather than passing through the daily landing flow.

Goal

Validate incoming files before they can affect downstream processing.

Student Activities

1. Create `data/landing/`, `data/raw/`, `data/rejected/`, `data/reference/` and `logs/`. Place `milano-grid.geojson` in `data/reference/` as a static reference asset.
2. Create one valid and one intentionally invalid training file. The invalid file should break something specific and named — a missing column, a malformed timestamp, or a negative activity value.
3. Implement `detect_files()`, `validate_schema()`, `validate_minimum_quality()` and `route_file()` for the daily activity CSVs. Detect using the pattern `sms-call-internet-mi-*.csv`. Do not treat the GeoJSON reference as a daily ingest candidate.
4. Write ingestion metadata for every file seen: `filename`, `status`, `row_count`, `reason`, `processed_at`.
5. Create a simple Airflow DAG for detect → validate → route → log.
6. Run both the valid and rejected paths and inspect the result in the Airflow UI.

Concepts

- landing / raw / rejected zones
- schema validation
- audit logging
- Airflow orchestration
- why reference data does not flow through ingestion

Expected Output

- raw validated file
- rejected file with a stated reason
- ingestion log
- Airflow DAG

Claude Usage for This Lab

Learner owns	Ingestion controls, and the distinction between operational input and reference data.
Primary AI mode	Claude Code — high-autonomy implementation.

Meaningful AI activity	Because learners already built a prior capstone, Claude may implement most of the ingestion boilerplate and its tests.
Recommended autonomy	High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Implement the agreed daily CSV ingestion design in ingestion/.

Required:
- detect new files in data/landing/ matching sms-call-internet-mi-*.csv
  (the -mi- is mandatory; a looser glob would admit files from a different
  city whose cell IDs collide numerically with Milan)
- validate the required raw columns
- validate minimum file quality
- route valid files to data/raw/
- route invalid files to data/rejected/ with a stated reason
- write audit metadata: filename, status, row_count, reason, processed_at
- generate tests for valid and invalid inputs

Do NOT treat data/reference/milano-grid.geojson as a daily landing file.
It is static reference data and is validated and managed separately.
```

Learner Validation & Discussion

- Trainer supplies one malformed file that is **not** covered by the generated tests.
- Learner diagnoses it and extends the validation and test coverage rather than special-casing it.
- Learner verifies the raw files are preserved unchanged after routing.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ A valid file lands in `data/raw/` and an invalid file lands in `data/rejected/`, both with an audit record.

☐ The audit record for the rejected file names the specific failed check.

☐ The GeoJSON in `data/reference/` is untouched by the ingestion run.

☐ Re-running the DAG on an already-processed file does not silently duplicate it — the behaviour is defined and logged, whether that is skip, warn or reject.

☐ Tests cover both the valid and the invalid path.

TRAINER FOCUS

Airflow orchestrates. Validation logic stays in reusable Python functions rather than being buried in DAG code — otherwise it cannot be tested and cannot be reused by SP7 or DE8.

↳ **Connects to:** DE3; DE8

DE3 — Orchestrate the Spark Processing Job

■ **Data: raw zone as populated by DE2** ◆**Autonomy: High after planning (DELEGATE)**

Business / Training Scenario

After ingestion succeeds, Airflow must invoke `telecom_pipeline.py` and publish the processed outputs.

Goal

Integrate the reusable PySpark job into the end-to-end pipeline.

Student Activities

1. Configure `telecom_pipeline.py` to read `data/raw/` and write `data/processed/` and `data/analytics/`.
2. Add an Airflow task that launches the Spark job, passing the reference path for `milano-grid.geojson`.
3. Make downstream tasks depend on Spark success.
4. Log Spark job start, end and status.
5. Test the empty-input path and the Spark-failure path.
6. Confirm analytics files are produced only after successful ingestion.

Concepts

- task dependencies
- Spark as processing engine, Airflow as orchestrator
- failure propagation
- why the orchestrator must not contain business logic

Expected Output

- DAG containing ingestion plus Spark task
- processed outputs
- execution logs

Claude Usage for This Lab

Learner owns	Airflow task boundaries and failure propagation.
Primary AI mode	Claude Code — plan, then implement.
Meaningful AI activity	Claude can implement the DAG after the learner approves the task graph.
Recommended autonomy	High after planning (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Explore ingestion/, spark/, airflow/ and the data zone layout.

Propose an Airflow task graph that:
- detects, validates and routes the daily CSV
- launches spark/telecom_pipeline.py
- passes data/reference/milano-grid.geojson as a static reference input
- publishes processed and analytics outputs only after successful
  processing
- logs start, end and status

Rules:
- Airflow must NOT duplicate cleaning or aggregation logic
- Spark business logic stays in telecom_pipeline.py
- downstream tasks must not run after a Spark failure

Show the task graph first. Implement only after I approve it.
```

Learner Validation & Discussion

- Learner explains what happens to each downstream task when Spark fails.
- Learner verifies that reference data is **read**, not re-ingested or copied per run.
- Learner confirms analytics outputs appear only after successful upstream tasks.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ A forced Spark failure leaves the analytics layer untouched and marks the DAG run failed.

☐ An empty raw folder produces a clean, understandable failure rather than an empty successful run.

☐ No cleaning or aggregation code exists inside the DAG file.

☐ The reference GeoJSON path is configuration, not a hardcoded string.

↳ **Connects to:** DE4; DE6; DE7

DE4 — Batch vs Streaming Decision Workshop

■ **Data: discussion lab — no data processing** ◆**Autonomy: Low (REVIEW)**

Business / Training Scenario

Different telecom workloads have different latency needs. Students classify operational scenarios and justify their choices rather than reaching for the newest tool.

Goal

Develop architecture judgement rather than tool memorization.

Student Activities

1. Classify: daily usage summary, hypothetical live activity events, billing report, hotspot alerts, executive dashboard refresh, and model training and scoring.
2. For each, document the source, arrival pattern, required latency, and the batch-or-streaming decision.
3. Identify where Kafka could conceptually enter the architecture, without changing this training dataset.
4. Discuss why these files are processed as batch even though real network activity is continuous.
5. Map batch to model training, and streaming to potential real-time scoring.
6. Present one decision where batch is the better engineering choice, and defend it.

Concepts

- batch versus streaming
- latency requirements
- event ingestion
- architecture trade-offs
- cost and operational complexity as first-class criteria

Expected Output

- decision matrix
- revised architecture with an optional streaming path drawn but not built

Claude Usage for This Lab

Learner owns	Make and defend architecture choices using latency and business needs.
Primary AI mode	Claude as adversarial reviewer.
Meaningful AI activity	Claude should challenge decisions rather than provide the initial answer. A team that asks Claude first has skipped the lab.
Recommended autonomy	Low (REVIEW)
Where Cowork fits	The decision matrix is a hand-over artifact — build it in Cowork.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
I will give you our batch-versus-streaming decisions for:
- daily activity summary
- hypothetical live activity events
- hotspot alerts
- executive dashboard refresh
- model training and scoring

Challenge each decision. Identify hidden assumptions about source arrival,
required latency, cost and operational complexity.

Remember: the project files are daily batch files containing hourly
observations. Do not pretend this dataset is a live stream.
```

Learner Validation & Discussion

- Learner defends at least one batch choice and one hypothetical streaming choice.
- Learner explains where Kafka could enter a real production architecture without changing this training dataset.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Every row of the decision matrix records required latency as a number or range, not as an adjective.

☐At least one decision is explicitly “batch, because streaming would add cost without adding value here”.

☐The revised architecture shows the streaming path as optional and unbuilt.

TRAINER FOCUS

Do not implement deep Kafka or Spark Streaming here unless separately scheduled. The objective is choosing correctly and being able to say why.

↳ **Connects to:** Future Kafka extension

DE5 — Storage Strategy & Data Zones

■ **Data: existing outputs from SP6** ◆**Autonomy: Medium (PAIR / REVIEW)**

Business / Training Scenario

The pipeline now produces multiple layers. Learners must decide which formats optimize auditability, processing and querying.

Goal

Choose practical storage formats for raw, processed and analytics data, and write the contract down.

Student Activities

1. Map the zones: landing = incoming daily CSVs; raw = immutable accepted CSVs; reference = `milano-grid.geojson`; processed = Parquet; analytics = warehouse tables and curated Parquet; `logs/` = audit and run history.
2. Define a date-partitioned processed directory structure, and a non-partitioned static location for the reference data.
3. Explain why raw is retained unchanged.
4. Compare append and overwrite semantics for each layer.
5. Define retention considerations for raw data and for logs.
6. Document the storage contract.

Concepts

- raw immutability
- Parquet
- partitioning
- retention
- lake and lakehouse thinking

Expected Output

- storage strategy table
- partitioned directory design
- written storage contract

Claude Usage for This Lab

Learner owns	The rationale for the landing, raw, reference, processed and analytics zones.
Primary AI mode	Claude as storage reviewer; Claude Code for the documentation and configuration update.
Meaningful AI activity	Claude can compare trade-offs, then update the project documentation and configuration after the learner chooses the convention.
Recommended autonomy	Medium (PAIR / REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review our current outputs and propose a storage strategy for:
- data/landing/   incoming daily CSVs
- data/raw/       immutable accepted CSVs
- data/rejected/
- data/reference/ milano-grid.geojson
- data/processed/ Parquet
- data/analytics/ curated Parquet and warehouse outputs
- logs/

Explain the trade-offs and recommend partitioning ONLY where it earns its
keep. Do not partition static reference data by date.
```

```
After I approve the strategy, update the repository documentation and
configuration to reflect it.
```

Learner Validation & Discussion

- Learner explains why raw is immutable, in terms of what becomes impossible if it is not.
- Learner explains why the GeoJSON is static reference data.
- Learner reviews the append and overwrite recommendations per layer rather than applying one rule everywhere.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Every zone has a stated format, a stated write mode and a stated retention position.

☐The reference zone is explicitly not date-partitioned.

☐`logs/` appears in the strategy, not just in the folder listing.

↳ **Connects to:** DE6

DE6 — Warehouse Modelling for Network Analytics

150 min ■ **Data: accumulated processed history** ◆**Autonomy: High (DELEGATE)**

Business / Training Scenario

Operations teams need repeatable queries for activity by time and grid, not direct scans of raw files.

Goal

Create an analytics-ready relational model from the processed activity summaries.

Student Activities

1. Identify the measures and the dimensions present in `hourly_grid_summary`.
2. Design `fact_network_activity` with grid and time keys and the activity measures.
3. Design `dim_time` and `dim_grid`. `dim_grid` holds `grid_id` plus optional centroid latitude and longitude and a geometry reference. Do **not** repeat the full Polygon geometry in `fact_network_activity`.
4. Create the SQL tables in PostgreSQL, MySQL or SQLite as appropriate to the environment.
5. Load the dimensions and the fact data from the Spark output, populating `dim_grid` **once** from the static Milan reference rather than once per hourly activity record.
6. Run queries for top grids, hourly trends and internet-heavy windows.
7. Add simple indexing on the common filter and join columns.

Concepts

- fact versus dimension
- star schema
- keys
- analytics queries
- indexing
- why geometry belongs in a dimension

Expected Output

- `fact_network_activity`
- `dim_time`
- `dim_grid`
- validated sample queries

Claude Usage for This Lab

Learner owns	Approve the minimal star schema and the treatment of geography.
Primary AI mode	Claude Code — high-autonomy SQL design and implementation.
Meaningful AI activity	Claude may generate the DDL and the load code after the learner approves the model.
Recommended autonomy	High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect hourly_grid_summary and the Milan grid reference.

Propose a minimal star schema supporting:
- grid activity drill-down
- hourly trends
- hotspot queries
- ML feature extraction

Use: fact_network_activity, dim_grid, dim_time.

Important:
- fact_network_activity holds grid/time keys and activity measures only
- dim_grid holds grid_id plus optional centroid latitude/longitude and a
  geometry reference
- do NOT repeat the full Polygon geometry in every fact row
- populate dim_grid from the static Milan reference ONCE
- dim_grid must have exactly 10000 rows if fully populated, or exactly the
  number of distinct observed grids if populated from activity

Show the model and DDL before applying it. After approval, generate the
DDL, the load logic and three validation queries.
```

Learner Validation & Discussion

- Learner reviews the keys and the indexes rather than accepting the defaults.
- Learner manually validates one aggregate query against the Spark output it came from.
- Learner explains why geometry belongs in or is referenced by the dimension, not repeated in the fact table.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`fact_network_activity` contains no geometry column.

☐`dim_grid` row count matches the distinct `grid_id` count in the source, with no duplicates on the key.

☐A hand-run SQL aggregate matches the equivalent Spark aggregate exactly.

☐Fact row count equals the `hourly_grid_summary` row count — no fan-out from the dimension join.

☐At least one index exists on the columns actually used for filtering and joining.

TRAINER FOCUS

Prefer names tied to actual activity. A `tower_load` table is misleading unless tower and capacity data is genuinely supplied, which it is not.

↳ **Connects to:** API1; API2; API6; ML2

DE7 — End-to-End Airflow Orchestration

■ **Data: full accumulated history, plus one held-back file to arrive live** ◆**Autonomy: High (DELEGATE + REVIEW)**

Business / Training Scenario

The operator wants the process to run from one trigger and produce an unambiguous success or failure result.

Goal

Automate the complete batch data flow with clear task ownership.

Student Activities

1. Define the tasks: ingest → validate → spark_process → load_warehouse → quality_check → notify.
2. Reuse the functions and jobs from previous labs rather than rewriting the logic.
3. Add task dependencies and failure behaviour.
4. Have `quality_check` write a machine-readable pipeline status record — this is the evidence source that API6, C3, C12 and C14 all consume, so it must be a file or a table, not a log line.
5. Add a lightweight success or failure notification or log entry.
6. Trigger the DAG with the held-back file and trace the data from landing to warehouse.
7. Document where to troubleshoot each failure type.

Concepts

- Airflow DAGs
- dependencies
- reusability
- observability
- pipeline status as a first-class data product

Expected Output

- working end-to-end DAG
- pipeline run log
- machine-readable pipeline status record
- troubleshooting map

Claude Usage for This Lab

Learner owns	The complete production flow, and knowing where each failure belongs.
Primary AI mode	Claude Code — high-autonomy integration.

Meaningful AI activity	Claude can act like a developer here, because the component patterns have already been learned.
Recommended autonomy	High (DELEGATE + REVIEW)
Where Cowork fits	The troubleshooting map is an operational runbook — a Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Explore the complete repository and build the end-to-end Airflow
orchestration using the EXISTING components:

  ingest -> validate -> spark_process -> load_warehouse
         -> quality_check -> notify

Requirements:
- reuse existing functions and jobs; never reimplement Spark or business
  logic inside the DAG
- use clear task names and structured logging
- treat milano-grid.geojson as static reference input
- quality_check must WRITE a machine-readable pipeline status record
  containing at minimum: run_id, run timestamp, per-task status, rows in,
  rows rejected, nulls handled, rows published, and the max timestamp now
  present in the analytics layer (which becomes AS_OF)
- produce an architecture-to-DAG mapping in docs/

Before editing, show a short cross-component plan. Then implement and run
the relevant tests.
```

Learner Validation & Discussion

- Learner triggers the DAG and traces one daily file through every zone.
- Learner answers: “Which module would you debug if `load_warehouse` fails?”
- Learner checks that the static grid reference is not copied through the daily ingestion path.
- Learner confirms the pipeline status record is readable by something other than a human — it will be served by API6.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐One trigger runs the whole flow and produces a single unambiguous success or failure.

☐The pipeline status record exists, is machine-readable, and contains the AS_OF value.

☐Re-running after a mid-pipeline failure does not corrupt or duplicate the analytics layer.

☐The troubleshooting map names a module for each failure type.

↳ **Connects to:** DE8; API6; Claude C3; Claude C14

DE8 — Network Pipeline Reliability Challenge

■ **Data: trainer-injected faulty files** ◆**Autonomy: High debugging / controlled modification (REVIEW then DELEGATE)**

Business / Training Scenario

The trainer injects multiple faults. Teams decide whether each should reject a file, warn, retry, fail the pipeline or continue — and then implement the controls.

Goal

Practise handling realistic failures, quarantines and safe reruns.

Student Activities

1. Test a missing daily file.
2. Test a duplicate file and a duplicate ingestion attempt.
3. Test malformed timestamps.
4. Test negative activity values.
5. Test an unexpected or missing column.
6. Test a partially corrupt file.
7. Simulate a Spark job failure.
8. For each case, record the expected action: REJECT, QUARANTINE, WARN, RETRY, FAIL or CONTINUE.
9. Demonstrate at least three implemented controls and one safe rerun.
10. Confirm each failure is reflected in the pipeline status record from DE7 — the assistant will read this later.

Concepts

- idempotency
- quarantine
- fail-fast versus warn
- retries
- operational runbooks

Expected Output

- failure handling matrix
- three implemented controls
- safe rerun demonstration
- pipeline status records showing each failure

Claude Usage for This Lab

Learner owns	Investigate the evidence before changing code, and choose the appropriate failure behaviour.
Primary AI mode	Claude Code — incident and debugging partner.
Meaningful AI activity	Claude may inspect logs and the repository deeply, but changes happen only after the learner reviews the diagnosis.
Recommended autonomy	High debugging / controlled modification (REVIEW then DELEGATE)
Where Cowork fits	The failure handling matrix is the operational deliverable — build it in Cowork.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
The Network Intelligence Airflow pipeline has failed.
```

```
Investigate the repository, the run logs and the recent input data, but
DO NOT immediately change any code.

Return:
  1. the most likely failure point
  2. the evidence that supports it
  3. the expected operational action:
     REJECT / QUARANTINE / WARN / RETRY / FAIL / CONTINUE
  4. a proposed correction
  5. the regression test that should be added

Possible faults include: missing daily CSV, duplicate ingestion, malformed
datetime, negative activity value, missing required column, partially
corrupt CSV, Spark failure.

Separate what you OBSERVED from what you INFERRED.

After I approve the diagnosis, implement the correction and the regression
test.
```

Learner Validation & Discussion

- Learner separates the evidence from Claude's hypothesis, and says which is which.
- Learner decides whether the correct response is retry, quarantine or failure — Claude proposes, the learner decides.
- Learner demonstrates a safe rerun that leaves the analytics layer correct.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐All seven fault types appear in the matrix with an assigned action.

☐At least three controls are implemented and demonstrated live.

☐A safe rerun is demonstrated and the analytics layer is verified correct afterwards, including the duplicate check on (`grid_id`, `timestamp`).

☐Each injected failure produces a distinguishable pipeline status record.

TRAINER FOCUS

This lab produces excellent evidence for the later Claude assistant: pipeline health and data-quality state become **tools and evidence** rather than vague narrative. Point this forward explicitly — C3 and C14 depend on what is built here.

↳ **Connects to:** API6; Claude C3; Claude C14

Phase 4 — FastAPI: Network Intelligence Service Layer

Purpose	Expose curated network intelligence as stable REST APIs that can be consumed by React, by the ML integration and by Claude tools.
Storyline	Warehouse and feature data → FastAPI → operational consumers.
Data scope	**The accumulated warehouse.** No endpoint reads a raw CSV.
Indicative duration	Approximately 9–10 hours
Autonomy note	Very High throughout. Backend patterns were learned in the prior capstone; learners own the contracts and the data correctness, not the boilerplate.

API1 — Network Summary Endpoint

■ **Data: analytics warehouse** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

The dashboard needs one endpoint that summarizes the current reporting window.

Goal

Expose the top-level NOC KPIs.

Student Activities

1. Create `GET /network/summary`.
2. Return `total_activity`, `active_grids`, `peak_hour` and `top_grid`.
3. Accept an optional `as_of` query parameter. When absent, default to the configured `AS_OF` — the maximum timestamp in the analytics layer. Never hardcode a date.
4. Use a Pydantic response model.
5. Query the warehouse and analytics layer rather than raw CSV.
6. Return clear 500 errors when the data source is unavailable, and include the effective `as_of` in every successful response.

Concepts

- FastAPI routing
- Pydantic
- service layer
- HTTP errors
- making a historical dataset behave like a live one, reproducibly

Expected Output

- working summary endpoint
- Swagger documentation

Claude Usage for This Lab

Learner owns	The API contract, and validating the results against the warehouse.
Primary AI mode	Claude Code — very high autonomy.

Meaningful AI activity	Claude can scaffold the route, the service, the response model and the tests using the existing project patterns.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect the analytics warehouse and the existing API structure.

Implement GET /network/summary returning:
- total_activity
- active_grids
- peak_hour
- top_grid
- as_of  (the effective reporting timestamp used for this response)

Requirements:
- optional as_of query parameter; when absent, default to the configured
  AS_OF value, which is MAX(timestamp) in the analytics layer
- never hardcode a date anywhere
- use a service layer and a Pydantic response model
- read the warehouse/analytics data, not raw CSV
- clear error handling and endpoint tests

Remember the metrics come from the grid/hour analytics layer AFTER
country-code aggregation.
```

Learner Validation & Discussion

- Learner runs one equivalent SQL query manually and compares the result field by field.
- Learner verifies the endpoint does not query raw country-code-level data.
- Learner calls the endpoint twice with an explicit `as_of` and confirms the response is identical both times.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Response matches a hand-run SQL query on every field.

☐Every response includes the effective `as_of`.

☐No date literal appears anywhere in the endpoint or service code.

☐Swagger renders the response model correctly.

↳ **Connects to:** API6; RE2; Claude C1; Claude C2

API2 — Grid Activity Drill-Down Endpoint

■ **Data: analytics warehouse** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

A NOC engineer selects a grid and needs its recent call, SMS, internet and total activity trend.

Goal

Serve the historical activity for one grid.

Student Activities

1. Create `GET /network/grid/{grid_id}`.
2. Add optional `date`, `hour` and `as_of` query parameters. Default the window to the trailing 24 hourly intervals ending at `AS_OF`.
3. Return the activity time series and the derived measures.
4. Return 404 for an unknown grid — and treat any `grid_id` outside 1–10000 as unknown.
5. Validate the results against SQL.

Concepts

- path versus query parameters
- 404 handling
- time-series JSON
- default windows derived from a shared AS_OF

Expected Output

- grid activity endpoint

Claude Usage for This Lab

Learner owns	The time-series contract and the grid-level semantics.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude can implement most of the endpoint and its tests.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add GET /network/grid/{grid_id}.

Requirements:
- optional date / hour / as_of filtering
- default window: the trailing 24 hourly intervals ending at AS_OF
- return the grid-level hourly activity time series
- include sms activity, call activity, internet activity and total_activity
- return 404 for an unknown grid, including any grid_id outside 1-10000
- validate inputs and add tests
- use the existing database and service patterns

Do NOT expose country-code rows as if they were separate grid
observations — one grid has one record per hourly interval.
```

Learner Validation & Discussion

- Learner compares one API response against the warehouse rows behind it.
- Learner checks the unknown-grid behaviour, including grid 0 and grid 10001.
- Learner confirms the returned series has one point per hour with no duplicates.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Grid 4821 returns exactly 24 points for the default window.

☐`grid_id` 0 and 10001 both return 404.

☐No timestamp appears twice in a single response.

☐Values match the warehouse exactly for a spot-checked hour.

↳ **Connects to:** RE3; Claude C2; Claude C3

API3 — Hotspot & Alert Endpoint

■ **Data: analytics warehouse + NP3 alert output** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

Operations needs a prioritized list rather than manually scanning ten thousand grids.

Goal

Serve the current high-activity areas and the rule-based alerts, with a contract that will survive the arrival of ML scores.

Student Activities

1. Create `GET /network/hotspots` and `GET /network/alerts`.
2. Allow `limit`, `severity` and `as_of` query parameters.
3. Initially serve the rule-based NP3 alerts.
4. Design the response shape so ML risk fields can be added later without breaking the React client.
5. Include `grid_id`, hourly `timestamp`, the relevant activity measures, status or severity, and a human-readable reason.

Concepts

- API contracts
- pagination and limits
- designing for a known future change
- evolution from rules to ML

Expected Output

- hotspot endpoint
- alert endpoint

Claude Usage for This Lab

Learner owns	The response contract, so rules can later evolve into ML scores without breaking clients.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude may build both endpoints and their tests.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add:
  GET /network/hotspots
  GET /network/alerts

Initially serve the NP3 rule-based alert data.

Design the response contract so that ML fields (risk_score, risk_level,
model_version) can be ADDED later without breaking the React client.
Include grid_id, hourly timestamp, the relevant activity measure(s),
status/severity and a human-readable reason.

Support limit, severity and as_of filters.

Do NOT label a hotspot as confirmed congestion anywhere in the response,
the field names or the documentation.
```

Learner Validation & Discussion

- Learner reviews the API response schema and can say which fields are additive-safe.
- Learner checks that the terminology remains hotspot, alert or operational attention throughout — including in field names.
- Learner confirms every `grid_id` returned exists in the Milan grid.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`limit` is respected exactly.

☐The word “congestion” appears nowhere in the response, the schema or the documentation.

☐Adding a nullable `risk_score` field later would not break an existing client — demonstrate this with a test.

☐Results are ordered deterministically, so the same request twice returns the same order.

↳ **Connects to:** RE4; ML6; Claude C2

API4 — Grid Feature Endpoint

■ **Data: ML feature table** ◆**Autonomy: High (DELEGATE + REVIEW)**

Business / Training Scenario

The prediction service and the Claude investigation agent both need a consistent feature vector for a grid.

Goal

Expose the ML-ready features for a selected grid and time window.

Student Activities

1. Create `GET /network/grid/{grid_id}/features`.
2. Return the exact ML2 feature set: `avg_activity`, `activity_growth`, `active_hours`, `peak_ratio`, `variability`, `internet_share`, plus `feature_timestamp`.
3. Define a stable Pydantic schema — this contract is consumed by ML5, RE5 and the Claude tools, and changing it later breaks three consumers.
4. Return data-quality status and feature freshness alongside the values.

5. Test the response against the stored feature table values.

Concepts

- feature serving
- data contract
- freshness metadata
- the API reads features, it does not compute them

Expected Output

- feature endpoint

Claude Usage for This Lab

Learner owns	Ensuring the API serves stored, reproducible features rather than recalculating them on the fly.
Primary AI mode	Claude Code — high autonomy.
Meaningful AI activity	Claude can implement the endpoint and its tests, but must use the stored feature table as the single source of truth.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Create GET /network/grid/{grid_id}/features.

Return exactly the ML2 feature set:
- avg_activity
- activity_growth
- active_hours
- peak_ratio
- variability
- internet_share
- feature_timestamp
- data-quality / freshness information

Use a stable Pydantic response model and add tests.

Do NOT duplicate feature-engineering logic inside the API. Read the stored
feature output produced by ml/features.py. If a feature is missing from
the stored table, return an explicit error rather than computing it here.
```

Learner Validation & Discussion

- Learner checks one feature response field by field against the stored feature table.
- Learner identifies where feature computation actually belongs, and confirms none of it has leaked into the API layer.
- Learner confirms the field names match ML2 exactly — a mismatch here silently breaks ML5 and RE5.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The six feature names match `ml/features.py` exactly, character for character.

☐No feature arithmetic exists in the API layer.

☐Response includes `feature_timestamp` and a freshness indicator.

☐A grid with no stored features returns a clear error rather than zeros.

↳ **Connects to:** ML2; ML4; ML5; Claude C2; Claude C14

API5 — Prediction Endpoint — Stub to Real Model

■ **Data: contract-first — no model yet** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

React and Claude development should not wait for the model implementation.

Goal

Define the prediction contract before ML is wired in.

Student Activities

1. Create `POST /network/predict-risk` with a Pydantic request model.
2. Return a stub `risk_score`, `risk_level`, `model_version` and `explanation_note` stating that the implementation is currently a stub.
3. Verify invalid inputs return meaningful validation errors.
4. After ML5, replace the stub with the trained model.
5. Keep the endpoint contract unchanged when that happens.

Concepts

- POST body validation
- API-first development
- stub-to-real integration
- contracts as a parallelization device

Expected Output

- prediction endpoint

Claude Usage for This Lab

Learner owns	The contract that ML5 must later preserve.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude can create the stub implementation and the validation tests.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add POST /network/predict-risk.

Define a stable request/response contract NOW, but use a stub prediction
implementation.
```

```
Response should include:
- risk_score
- risk_level
- model_version
- explanation_note indicating the implementation is currently a stub

Add request validation and tests.

Keep the contract suitable for ML5 to replace the implementation WITHOUT
requiring any React change.
```

Learner Validation & Discussion

- Learner reviews the contract before accepting it, thinking specifically about what ML5 will need.
- Learner confirms invalid inputs return predictable, documented validation errors.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`model_version` is present in the stub response and clearly marked as a stub.

☐A request missing a required field returns 422 with a readable message.

☐The contract is written down in `docs/` so ML5 can be checked against it.

↳ **Connects to:** RE5; ML5; Claude C2

API6 — Operational Support Endpoints — Pipeline Status & Grid Location

■ **Data: pipeline status record from DE7 + `dim_grid`** ◆**Autonomy: High (DELEGATE + REVIEW)**

Business / Training Scenario

The Claude Network Operations Assistant will be asked two kinds of question that none of the previous endpoints can answer: *"can I trust this data right now?"* and *"where is this grid?"*. Both need a real service behind them, not a narrative from the model.

Goal

Expose pipeline health and grid geography as first-class, tool-callable evidence sources.

Student Activities

1. Create `GET /pipeline/status`. Read the machine-readable status record written by DE7's `quality_check` task. Return the last run identifier and timestamp, per-task status, rows in, rows rejected, nulls handled, rows published, the current `AS_OF`, and a freshness indicator saying how old the analytics layer is.
2. Include a top-level `healthy` boolean so a caller can branch on one field, and a `reasons` list when it is false.
3. Create `GET /network/grid/{grid_id}/location`. Read `dim_grid` and return `grid_id`, centroid latitude and longitude, and a reference to the polygon. Do not return the full Polygon geometry in this endpoint.
4. Optionally add `GET /network/grid/{grid_id}/neighbours` returning nearby grid IDs by centroid distance — this is what makes "are the surrounding cells also elevated?" answerable at C2.
5. Add tests, including a test that `/pipeline/status` correctly reports unhealthy after a deliberately failed run.
6. Document both endpoints as the sanctioned evidence sources for the Claude phase.

Concepts

- operational metadata as an API
- health endpoints versus status endpoints
- giving an agent a way to check its own evidence
- why an assistant must never narrate pipeline health from memory

Expected Output

- `GET /pipeline/status`
- `GET /network/grid/{grid_id}/location`
- optional neighbours endpoint
- tests including an unhealthy-state test

Claude Usage for This Lab

Learner owns	Deciding what an operations assistant would need to answer "can I trust this?" — and confirming the endpoint actually answers it.
Primary AI mode	Claude Code — high autonomy.
Meaningful AI activity	Claude can implement both endpoints. The learner defines what "healthy" means for this pipeline, because that is a judgement about the business, not about code.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect the pipeline status record written by the DE7 quality_check task,
and the dim_grid table.

Implement two endpoints:

1. GET /pipeline/status
   - read the DE7 status record; do not recompute anything
   - return: last run_id, run timestamp, per-task status, rows in,
     rows rejected, nulls handled, rows published, current AS_OF, and
     an analytics-layer freshness indicator
   - include a top-level healthy boolean and a reasons list when false

2. GET /network/grid/{grid_id}/location
   - read dim_grid
   - return grid_id, centroid latitude, centroid longitude and a polygon
     reference
   - do NOT return full Polygon geometry from this endpoint

Add tests, including one that asserts /pipeline/status reports unhealthy
after a deliberately failed pipeline run.

These two endpoints are the sanctioned evidence sources for the Claude
assistant in Phase 7. Document them in docs/ as such.
```

Learner Validation & Discussion

- Learner defines what “healthy” means for this pipeline and can defend the definition.
- Learner deliberately fails a DAG run and confirms `/pipeline/status` reports it, rather than reporting stale success.
- Learner verifies the centroid returned for a known grid sits where they expect it on a map.
- Learner confirms the endpoint reads the DE7 record rather than recomputing status independently — two sources of truth for health is worse than none.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`/pipeline/status` returns `healthy: false` with a populated `reasons` list after an injected DE8 failure.

☐`/pipeline/status` exposes the same `AS_OF` value that API1 uses — verify they agree.

☐`/network/grid/4821/location` returns a centroid inside the Milan bounding box.

☐The location endpoint returns no Polygon geometry.

☐Both endpoints are documented as the Claude-phase evidence sources.

TRAINER FOCUS

This lab exists because four later labs — C3, C12, C14 and the capstone — ask the assistant to verify pipeline health, and one asks it to locate a grid. Without these endpoints the assistant has nothing to call and will either refuse or, worse, invent an answer. Make the connection explicit when introducing the lab: **you are building the evidence the assistant will be graded on using.**

↳ **Connects to:** RE4; Claude C2; Claude C3; Claude C12; Claude C14; Capstone

Phase 5 — React: Lightweight NOC Dashboard

Purpose	Build only enough frontend to make the network intelligence visible and interactive. React remains an API consumer, not a separate frontend engineering track.
Storyline	FastAPI → operational cards, grid drill-down, hotspots and map, predictive risk view.
Data scope	**API responses only.** The dashboard never reads the warehouse or a file directly, with one deliberate exception: the static GeoJSON at RE4.
Indicative duration	Approximately 8–9 hours
Autonomy note	Very High. The learning is full-stack integration and map/API correctness, not component craft.

RE1 — Set Up the NOC Dashboard

■ **Data: API only** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

Learners establish the UI shell and confirm one API call works end to end.

Goal

Create the React app and configure connectivity to FastAPI.

Student Activities

1. Create the app and the API base configuration.
2. Configure CORS on FastAPI.
3. Fetch `/network/summary` on load.
4. Implement loading, success and error states.
5. Create simple navigation between the dashboard pages.

Concepts

- React project structure
- fetch or axios
- `useEffect`
- CORS

Expected Output

- working app shell

Claude Usage for This Lab

Learner owns	API configuration and runtime errors.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude may scaffold the frontend shell and the API client pattern.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Scaffold the frontend for the Network Operations dashboard.

Configure:
- API base URL from environment configuration, not hardcoded
- basic navigation
- a call to /network/summary
- loading / success / error states
- CORS expectations on the FastAPI side
- minimal styling

Use simple reusable components. Do not introduce unnecessary frontend
frameworks or state libraries.
```

Learner Validation & Discussion

- Learner runs the app and resolves the environment and configuration issues themselves.
- Learner can explain where the API base URL is configured and how it would change for another environment.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The app loads and renders a value that came from the API.

☐Stopping the API produces a visible error state rather than a blank screen or a console-only failure.

☐No API URL is hardcoded in a component.

↳ **Connects to:** RE2

RE2 — Network Overview Page

■ **Data: `/network/summary`** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

The first screen should answer four questions: how much activity, when is the peak, how many grids are active, and which grid is currently highest.

Goal

Display the top-level NOC KPIs.

Student Activities

1. Call `/network/summary`.
2. Display four metric cards.
3. Show the `as_of` value returned by the API as the reporting timestamp — do not display the browser clock, which would be misleading on a historical dataset.
4. Show a small status banner when the API is unavailable.
5. Keep the styling intentionally simple.

Concepts

- API consumption
- state
- conditional rendering
- displaying the data's time, not the user's time

Expected Output

- NOC overview page

Claude Usage for This Lab

Learner owns	Validating the UI-to-API data mapping.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude can build the page from the existing summary contract.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build the Network Overview page using GET /network/summary.

Show:
- total activity
- active grids
- peak hour
- top grid
- the as_of reporting timestamp RETURNED BY THE API

Do not display the browser clock as the reporting time — this is a
historical dataset and the API tells us what "now" means.

Keep components small and reusable and add a visible API-unavailable
state.
```

Learner Validation & Discussion

- Learner compares the displayed values against the raw API response.
- Learner verifies that “total activity” is presented as an activity indicator, not as literal traffic volume — check the card label wording.
- Learner confirms the displayed timestamp is the API’s `as_of`, not `new Date()`.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐All four values match the API response exactly.

☐The reporting timestamp shown is the API `as_of`.

☐No card label implies megabytes, message counts or congestion.

↳ **Connects to:** RE3

RE3 — Grid Explorer

■ **Data: `/network/grid/{grid_id}`** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

A user enters or selects a grid and views its recent activity by time.

Goal

Allow operational drill-down by grid.

Student Activities

1. Create a grid input or search control.
2. Call `GET /network/grid/{grid_id}`.
3. Render an activity table or a simple time-series chart.
4. Display call, SMS, internet and total activity as separate series.
5. Handle an unknown grid gracefully.

Concepts

- controlled inputs
- dynamic endpoints
- chart and table rendering

Expected Output

- grid explorer page

Claude Usage for This Lab

Learner owns	Dynamic API calls and hourly time-series display.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude may build the component and the chart integration.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add a Grid Explorer page.

The user selects or enters a grid_id.
Call GET /network/grid/{grid_id}.

Display:
- hourly SMS activity
- hourly call activity
- internet activity
- total_activity
- a simple time-series chart or table

Handle unknown grids and API errors cleanly. Remember internet activity
often dominates total_activity, so consider whether a single shared axis
is readable.
```

Learner Validation & Discussion

- Learner verifies the mapping of API fields to chart or table fields.
- Learner checks one grid against the warehouse-derived data.
- Learner notices whether internet activity dominates the chart scale, and decides deliberately how to handle it.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Entering grid 4821 renders 24 points.

☐Entering grid 99999 shows a clean not-found state, not a crash.

☐The four series are individually distinguishable.

↳ **Connects to:** RE4; Claude C2

RE4 — Hotspots, Alerts and the Milan Map

■ **Data: `/network/hotspots`, `/network/alerts`, and a static copy of `milano-grid.geojson`** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

The NOC team needs a ranked list of high-activity grids and current alerts, and needs to see where those grids actually are.

Goal

Render prioritized operational attention areas, geographically.

Student Activities

1. Call `/network/hotspots` and `/network/alerts`.
2. Load a read-only static copy of `milano-grid.geojson` from the frontend `public/reference/` area or an equivalent static route. Fetch it **once** and hold it in state — it must not be re-fetched on every interaction.
3. Render ranked rows with grid, activity, alert or risk status, and hourly timestamp.
4. Add limit and severity filtering.
5. Render the Milan grid polygons and join hotspot/alert status to the map by `grid_id`.
6. Visually distinguish NORMAL, ATTENTION and HIGH in both the ranked table and the map.
7. Allow a selected or highlighted polygon to open the Grid Explorer for that grid.

Concepts

- lists and tables
- query parameters
- conditional styling
- joining API state to static geographic reference data
- rendering budget as a design constraint

Expected Output

- hotspot and alert page with a Milan geographic hotspot map

Claude Usage for This Lab

Learner owns	Validating how operational status is joined to the static geographic reference data.
Primary AI mode	Claude Code — very high autonomy, including the map implementation.
Meaningful AI activity	Claude can implement the map and the UI, because the learning objective is integration, not GIS or frontend mastery.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build the Hotspots & Alerts page.

Use:
- GET /network/hotspots
- GET /network/alerts
- a read-only static copy of milano-grid.geojson

PERFORMANCE CONSTRAINT — read this before choosing an approach:
milano-grid.geojson contains 10,000 Polygon features and is about 3.2 MB.
Rendering all 10,000 as individual SVG paths will make the page visibly
unresponsive. Propose a rendering strategy BEFORE implementing. Acceptable
strategies include: rendering only the grids returned by the API for the
selected window; rendering the top N hotspots as polygons over a plain
base map; or using a canvas/WebGL renderer rather than SVG.
Fetch the GeoJSON ONCE and keep it in state.

Required:
- ranked hotspot / alert table
- hourly timestamp
- activity / risk status
- severity filter
- Milan grid polygon map
- join API hotspot/alert data to map features by grid_id, using
  properties.cellId from the GeoJSON — NOT the top-level feature id,
  which is 0-based and would shift every cell by one
- visual states for NORMAL / ATTENTION / HIGH
- clicking or highlighting a polygon opens or links to the Grid Explorer

Do not invent zone names, and do not display "congested" anywhere.
```

Learner Validation & Discussion

- Learner verifies the map uses the actual grid geometry rather than plotted points.
- Learner checks the join key between the GeoJSON and the API `grid_id` — and confirms it is `properties.cellId`.
- Learner confirms the GeoJSON is loaded once as static reference data and is not sent repeatedly from the warehouse.
- Learner interacts with the map and confirms it stays responsive.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The map remains interactive — panning and zooming do not visibly stall.

☐The GeoJSON is fetched exactly once per page load, verifiable in the network tab.

☐**The highlighted polygon for the top hotspot is the correct cell** — cross-check its centroid against the `/network/grid/{id}/location` response from API6.

☐Clicking a polygon navigates to the Grid Explorer for that same grid.

☐NORMAL, ATTENTION and HIGH are distinguishable without relying on colour alone.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

Join the map to the API on `properties.cellId`. The top-level `id` on each feature is 0-based and will shift every cell by one — the map renders, the colours appear, and every hotspot is drawn on the wrong square.

The API6 location endpoint exists partly so this can be verified: pull the centroid for the top hotspot from the API and confirm it matches the polygon being highlighted. This is the only reliable check.

TRAINER FOCUS

This lab is the visible payoff of SP4. A learner should be able to trace the whole chain out loud: raw `CellID` → canonical `grid_id` → GeoJSON `properties.cellId` → Polygon → highlighted NOC map. Ask them to do exactly that.

↳ **Connects to:** ML6; Claude C2

RE5 — Predictive Risk View

■ **Data: `/network/predict-risk`** ◆**Autonomy: Very High (DELEGATE)**

Business / Training Scenario

After the model is deployed, the user can request a risk or anomaly score for a grid and time window.

Goal

Connect the UI to the prediction service, without implying more certainty than the model has.

Student Activities

1. Submit feature values or a selected grid to `POST /network/predict-risk`.
2. Display the risk score, the risk level and the model version.
3. Show the model output visually separate from any narrative explanation.
4. Add a placeholder “Explain with AI” action for the later Claude phase.

Concepts

- POST integration
- model output display
- separation of prediction and explanation

Expected Output

- predictive risk page

Claude Usage for This Lab

Learner owns	Keeping model output separate from the later Claude explanation.
Primary AI mode	Claude Code — very high autonomy.
Meaningful AI activity	Claude may build the prediction UI and the placeholder action.
Recommended autonomy	Very High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build the Predictive Risk view against POST /network/predict-risk.
```

```
Display:
- risk score
- risk level
- model version

Keep the model output visually separate from any narrative explanation —
a user must be able to tell at a glance which number came from the model
and which words came from an LLM.

Add an "Explain with AI" placeholder action for the later Claude Network
Operations Assistant phase. Do not implement the assistant yet.
```

Learner Validation & Discussion

- Learner verifies the request and response mapping.
- Learner can explain the difference between a model score and an LLM explanation, and point to where that difference is visible in the UI.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`model_version` is displayed, not hidden.

☐Model output and narrative space are visually distinct regions.

☐No label anywhere presents the risk score as a confirmed fault or as congestion.

TRAINER FOCUS

Do not let the UI imply the ML score is a confirmed network fault. It is an attention signal. The wording on this page is assessed.

↳ **Connects to:** Claude C1; Claude C2

Phase 6 — Machine Learning: Lightweight Network Activity Intelligence

Purpose	Add simple, explainable predictive and anomaly intelligence without turning the programme into a mathematically heavy ML course.
Storyline	Historical summaries → engineered features → simple model and baseline → risk / anomaly score → API → dashboard.
Data scope	**The accumulated processed history.** Fourteen days recommended, ten the absolute minimum — below that the chronological split has too little history on either side to be meaningful.
Indicative duration	Approximately 11–12 hours
Autonomy note	Deliberately reduced for ML1–ML3, where learners must own problem definition, feature meaning and time-aware validation. It rises again for operationalization in ML5–ML6.

ML1 — Frame the Operational ML Problem

■ **Data: design lab** ◆**Autonomy: Advisor only (REVIEW)**

Business / Training Scenario

Learners choose one primary training problem and, just as importantly, decide what the model is forbidden from claiming. They explicitly avoid asserting real congestion without capacity data.

Goal

Define what the model is allowed to claim, what evidence it uses, and what it is actually predicting.

Student Activities

1. Compare three candidate problems: high-activity risk, anomalous activity, and activity drop.
2. Select one primary problem for the model.
3. Define the prediction unit: a grid plus a time window.
4. **Define the target as a future window.** The label describes the interval at `t+1`; the features describe the trailing window ending at `t`. See the trap below — this is the difference between a real prediction problem and a restatement of a threshold.
5. Define the target strategy precisely. If synthetic threshold labels are used, document them as training proxies and state the threshold.
6. Define the business action: “investigate”, never “the network is congested”.
7. Identify the data leakage risks, and state how the `t` / `t+1` boundary prevents each one.

Concepts

- problem definition
- label and target design
- proxy labels
- the difference between describing and predicting
- business action
- data leakage

Expected Output

- ML problem statement

- feature and target sheet with the `t` / `t+1` boundary written down

Claude Usage for This Lab

Learner owns	Define the prediction problem and defend what the data can and cannot support.
Primary AI mode	Claude as critical reviewer.
Meaningful AI activity	Learners decide first. Claude challenges overclaiming, target design and leakage. Claude must not choose the problem.
Recommended autonomy	Advisor only (REVIEW)
Where Cowork fits	The ML problem statement is a document that will be read by people who were not in the room — a Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review our proposed ML problem for this network activity project.

Available data represents grid-level communication activity over hourly
intervals. We do NOT have capacity, throughput, latency, packet loss or
radio-utilization data.

Challenge whether our proposed target can legitimately be called
congestion.

Evaluate these alternatives:
- high-activity risk
- unusual / anomalous activity
- activity drop

CRITICAL — check our target for circularity. We intend to predict the
state of a grid at interval t+1 using only features computed from the
trailing window ending at t. Tell us plainly whether any feature we have
proposed would leak information from t+1, and whether our label could be
trivially reconstructed from our features. If the model could score near
perfectly by restating a threshold it already has, say so directly.

Also identify:
- the prediction unit
- proxy-label limitations
- remaining leakage risks
- what operational action should follow a positive result

Do NOT choose the problem for us. Critique our proposal.
```

Learner Validation & Discussion

- Learner writes the final ML problem statement themselves.
- Learner explicitly states that a positive result means “investigate”, not “the network is congested”.
- Learner identifies at least one leakage risk and explains how the `t` / `t+1` boundary addresses it.
- Learner can state, in one sentence, what the model knows at prediction time that a simple threshold does not.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The problem statement names the prediction unit, the label definition and the exact time boundary between features and label.

☐The label refers to a **future** interval. A label computed over the same window as the features fails this lab.

☐A non-goals list exists and includes congestion, capacity and throughput.

☐The business action is written as an instruction to a human, not as a conclusion.

⚠ KNOWN TRAP — BRIEF LEARNERS BEFORE THEY START

The obvious formulation is circular. If the label is "`total_activity` exceeded a threshold in this window" and the features are `avg_activity`, `peak_ratio`, `activity_growth` and `variability` **computed over that same window**, then the model is being asked to predict a number from itself. It will score near-perfectly, the learners will believe the pipeline works, and the entire evaluation discussion at ML3 becomes meaningless because the answer is always ~99%.

The fix costs nothing and is the whole lesson: **features from the trailing window ending at `t`, label describing `t+1`.** Now the model has to generalize, precision and recall become informative, class imbalance becomes real, and the chronological split at ML3 finally matters.

If a team reports 99% accuracy at ML3, do not congratulate them. Send them back to this lab.

TRAINER FOCUS

This lab protects the project from overclaiming in two directions at once — overclaiming about the network (congestion), and overclaiming about the model (a tautology reported as skill). Both are common, and both are more damaging than a mediocre score honestly reported.

↳ **Connects to:** ML2

ML2 — Engineer Network Activity Features

■ **Data: accumulated history, 14 days recommended** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

Features should capture level, trend, peakiness and variability without complex signal processing.

Goal

Create a small, understandable feature set from the Spark and warehouse summaries — computed strictly from the past.

Student Activities

1. Fix the feature window convention first: every feature for a prediction at `t+1` is computed from the trailing window **ending at `t`**, inclusive of `t` and nothing after it. Write this down before writing code.
2. Create `avg_activity` over the recent trailing window.
3. Create `activity_growth` — the recent window against a prior baseline window.
4. Create `active_hours` — the count of hours in the window with activity above zero.
5. Create `peak_ratio` = peak ÷ average over the window.
6. Create `variability` using standard deviation or a coefficient-of-variation style proxy.
7. Create `internet_share` = `internet_activity` ÷ `total_activity`.
8. Persist the feature table with `grid_id` and `feature_timestamp`, where `feature_timestamp` is `t` — the last interval the features are allowed to see.

9. Write one test that would fail if any feature used data from after `feature_timestamp`.

Concepts

- feature engineering
- rolling and baseline comparisons
- reproducibility
- leakage prevention as a testable property, not a discussion

Expected Output

- `network_feature_table` with `grid_id` and `feature_timestamp`
- a leakage test

Claude Usage for This Lab

Learner owns	The operational meaning and the time window behind every feature.
Primary AI mode	Claude as feature reviewer; Claude Code as implementation partner.
Meaningful AI activity	Claude can implement approved features after explaining them. It must not choose the windows.
Recommended autonomy	Medium–High (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect the stored grid/hour analytics data and propose a minimal feature
set for a simple operational risk / anomaly model.

WINDOW RULE — apply this to every feature without exception:
features for a prediction about interval t+1 are computed ONLY from the
trailing window ending at t. Nothing after t may be read. feature_timestamp
records t.

Candidate features:
- avg_activity over a recent trailing window
- activity_growth against a prior baseline window
- active_hours
- peak_ratio
- variability
- internet_share

For each feature, explain:
- formula / logic
- lookback window
- operational meaning
- how the window rule prevents leakage for that specific feature

Do NOT implement until I approve the feature definitions and the windows.

After approval, implement them in ml/features.py, persist
network_feature_table with grid_id + feature_timestamp, and add a test
that FAILS if any feature reads data after feature_timestamp.
```

Learner Validation & Discussion

- Learner approves the lookback and baseline window for each feature, and can say why that length.
- Learner checks the feature values for one grid by hand against the underlying hourly rows.
- Learner ensures future data is not used to calculate past features — and confirms the leakage test actually fails when deliberately broken.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The six feature names match exactly what API4 serves — character for character.

☐Every row carries a `feature_timestamp`.

☐The leakage test exists, passes on the real implementation, and **fails** when a feature is deliberately allowed to see `t+1`. Demonstrate both.

☐Hand-checking one grid reproduces `avg_activity` and `peak_ratio` exactly.

☐No feature is undefined or infinite for a grid with zero activity — division by zero is handled explicitly.

↳ **Connects to:** ML3; ML4; API4

ML3 — Train a Simple Risk Classifier

■ **Data: accumulated history, 14 days recommended** ◆**Autonomy: Medium (PAIR)**

Business / Training Scenario

Using the engineered feature table, learners train a simple classifier for the selected proxy risk label — and then interpret it honestly.

Goal

Demonstrate the standard ML workflow using an interpretable baseline model.

Student Activities

1. Sort the feature data by time and perform a **chronological** train/test split: earlier periods train, later periods test. Never a random split.
2. Record and report the earliest and latest timestamp in each of the train and test sets.
3. Train Logistic Regression or a Decision Tree.
4. Evaluate accuracy **plus** precision, recall and the class balance. Report the base rate — the proportion of positive labels — alongside accuracy, because accuracy is meaningless without it.
5. Inspect the coefficients or the feature importances and check they are operationally plausible.
6. Compare the predictions against the rule-based NP3 alerts and characterize where they disagree.
7. Write three observations about where the model adds value and where it does not.

Concepts

- chronological train/test split for time-based data
- classification
- precision and recall
- base rate as context for accuracy
- interpretability

Expected Output

- trained model

- evaluation report including the base rate
- comparison against NP3 alerts

Claude Usage for This Lab

Learner owns	Algorithm choice, chronological validation and metric interpretation.
Primary AI mode	Claude as ML reviewer; Claude Code for implementation after approval.
Meaningful AI activity	Claude may implement the baseline model, but the learner owns the time split and the interpretation of the metrics.
Recommended autonomy	Medium (PAIR)
Where Cowork fits	The evaluation report is a written artifact for a reviewer — a Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review network_feature_table and propose either Logistic Regression or a
Decision Tree as a simple baseline.

Important:
- this is time-based data
- use a CHRONOLOGICAL train/test split: earlier periods train, later
  periods test
- do NOT randomly mix neighbouring time observations across train and test
- the label describes interval t+1; the features describe the trailing
  window ending at t

Explain:
- why the selected algorithm fits this training objective
- the exact split strategy and the resulting date ranges
- precision versus recall for this operational use case
- the class balance and the base rate of the positive label
- any remaining leakage

Report the BASE RATE alongside accuracy. If accuracy is close to the base
rate, say so. If accuracy is near-perfect, treat that as a symptom of
leakage or a circular label and investigate before reporting it.

Show the training plan before coding. After approval, implement training
and evaluation, and compare the predictions with the NP3 rule alerts.
```

Learner Validation & Discussion

- Learner verifies the earliest and latest timestamps in the train and test sets and confirms they do not overlap.
- Learner explains why random splitting makes evaluation unrealistically optimistic on time-series data.
- Trainer asks: *"Is 85% accuracy enough?"* The learner must discuss precision, recall, the base rate and the operational cost of a false positive rather than deferring to Claude.
- Learner characterizes at least one case where the model and the NP3 rule disagree, and says which they would trust.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Train and test date ranges are reported and do not overlap.

☐The base rate of the positive class is reported next to accuracy.

☐Precision and recall are reported separately, not merged into a single figure.

If accuracy exceeds roughly 95%, the lab is not signed off until the cause is investigated. On this data and this formulation that indicates leakage or a circular label, not a good model.

☐Three written observations exist, and at least one names a limitation.

TRAINER FOCUS

Keep the algorithm choice deliberately simple. The value is in the pipeline and the interpretation, not in benchmark performance. Because the feature table is time-based, a chronological split is required — the evaluation should reflect learning from the past and testing on a later period.

Trainer check: A chronological split is a required dataset-alignment decision. Do not accept a default `train_test_split` without discussion. And do not accept a suspiciously perfect score without investigation — that is the ML1 trap resurfacing.

↳ **Connects to:** ML4; ML5

ML4 — Add an Anomaly Baseline

90 min ■ Data: accumulated history — this is where the hour-of-day baseline finally becomes possible ◆Autonomy: High (DELEGATE + REVIEW)

Business / Training Scenario

A second, independent mechanism compares current activity against historical baseline behaviour — and it can now do what NP3 could not.

Goal

Teach abnormal-behaviour thinking without complex unsupervised ML.

Student Activities

1. Compute the historical mean or median per grid **and hour-of-day bucket**. This is the baseline NP3 could not build from a single day, and it is now possible because the pipeline has accumulated history.
2. **Reuse the NP3 baseline function rather than writing a second one.** Generalize it to accept a bucketing key, so the within-day and hour-of-day baselines share one implementation.
3. Compute the deviation from the baseline.
4. Define an anomaly score using a simple standardized or percentage deviation.
5. Flag both unusually high and unusually low activity, and keep the direction as a field.
6. Compare the anomaly flag against the ML3 classifier output and the NP3 rule alerts, and explain the disagreements.
7. Store both the score and a human-readable reason.

Concepts

- baseline anomaly detection
- deviation
- explainable scoring
- why three mechanisms disagreeing is informative rather than a defect

Expected Output

- `network_anomaly_scores`
- a three-way comparison against NP3 and ML3

Claude Usage for This Lab

Learner owns	Simple historical-deviation logic, and comparing it against the rules and the classifier.
Primary AI mode	Claude Code — high autonomy.
Meaningful AI activity	The method is intentionally simple and explainable, so Claude can implement most of it.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build a simple explainable anomaly scorer using historical behaviour for
each grid and comparable hour-of-day bucket.

First inspect the NP3 rule-based alert module. It already contains a
baseline function built for a single day. Generalize THAT function to
accept a bucketing key rather than writing a second implementation, so the
within-day and hour-of-day baselines share one code path.

Avoid advanced unsupervised algorithms.

Consider:
- historical mean or median per grid and hour-of-day
- percentage deviation
- optionally standardized deviation

Return:
- anomaly_score
- direction (high or low)
- baseline value
- current value
- a human-readable reason

Then compare the output against:
- the NP3 rule alerts
- the ML3 classifier output
and summarize where and why they disagree.
```

Learner Validation & Discussion

- Learner inspects at least two high and two low anomaly cases against the underlying data.
- Learner can explain why a rule alert, a classifier result and an anomaly score can legitimately disagree.
- Learner confirms the baseline function is shared with NP3 rather than duplicated.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The hour-of-day baseline uses more than one day of history — verify the bucket counts.

☐One baseline implementation exists in the codebase, not two.

☐Both high and low anomalies are produced, with direction recorded.

☐A three-way comparison table exists and at least one genuine disagreement is explained.

TRAINER FOCUS

Draw the line back to NP3 explicitly. The within-day baseline could not tell “normally quiet at 03:00” from “dropped”. This one can — because the pipeline the learners built has been accumulating history. That is the payoff of Phase 3, made concrete.

↳ **Connects to:** ML5; ML6; Claude C1; Claude C3

ML5 — Operationalize the Model through FastAPI

■ **Data: trained model + feature table** ◆**Autonomy: Very High (DELEGATE + REVIEW)**

Business / Training Scenario

The API contract already exists. Learners now wire the trained model and the anomaly scorer behind it, without changing the contract.

Goal

Replace the prediction stub with real risk and anomaly logic.

Student Activities

1. Load the trained model safely at service startup, with a clear failure if the artifact is missing.
2. Validate the incoming features against the ML2 schema.
3. Return `risk_score`, `risk_level`, `model_version` and `feature_timestamp`.
4. Optionally return the top contributing feature names when the model supports it.
5. Retest the React prediction page without changing the frontend contract.

Concepts

- model serving
- version metadata
- inference contract
- replacing an implementation without touching its consumers

Expected Output

- real `POST /network/predict-risk` endpoint

Claude Usage for This Lab

Learner owns	Preserving the existing API contract while replacing the stub safely.
Primary AI mode	Claude Code — very high autonomy, plan first.
Meaningful AI activity	An integration task rather than a new ML concept, so Claude may perform most of the change after presenting a cross-file plan.
Recommended autonomy	Very High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Inspect:
- ml/
- the existing POST /network/predict-risk stub
- API models and services
- tests
- the React prediction consumer

Create a plan to replace the stub with the trained model and anomaly logic
WITHOUT changing the API contract.

Add:
- safe model loading at service startup, failing clearly if the artifact
  is missing
- input validation against the ML2 feature schema
- model_version
- feature_timestamp
- tests

Do NOT modify the React client unless the existing contract is genuinely
insufficient — and if you believe it is, say why before changing anything.

Show the cross-file plan before editing, then implement and run the
regression tests.
```

Learner Validation & Discussion

- Learner reviews the multi-file diff.
- Learner runs React → FastAPI → ML → React end to end.
- Learner checks that the model output remains separate from any later Claude narrative.
- Learner confirms the React code was not modified.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The React prediction page works with **zero** frontend changes.

☐A missing model artifact produces a clear startup failure, not a silent fallback to the stub.

☐`model_version` reflects the actual trained model, not the stub placeholder.

☐All API5 tests still pass unchanged.

↳ **Connects to:** RE5; ML6; Claude C2

ML6 — Batch Score All Grids

■ **Data: full accumulated history** ◆**Autonomy: Very High (DELEGATE + REVIEW)**

Business / Training Scenario

After each processed batch, all grid and time windows are scored and a current risk table is published.

Goal

Integrate ML into the data engineering flow as a repeatable pipeline stage.

Student Activities

1. Read the latest feature table.
2. Run the risk and anomaly scoring in batch.
3. Write `network_risk_scores` with `grid_id`, `timestamp`, `risk_score`, `risk_level` and `model_version`.
4. Add a scoring task to the Airflow DAG, positioned after feature generation and before the quality check.
5. Validate that `/network/hotspots` and `/network/alerts` can surface the model scores through the additive fields designed at API3.
6. Produce a top-twenty operational attention report.

Concepts

- batch inference
- ML and DE integration
- model metadata
- additive API evolution in practice

Expected Output

- `network_risk_scores`
- Airflow scoring task
- top-risk report

Claude Usage for This Lab

Learner owns	How ML becomes a repeatable production pipeline stage.
Primary AI mode	Claude Code — very high autonomy across `ml/`, `airflow/`, `api/` and `tests/`.
Meaningful AI activity	Claude can implement the cross-component integration after presenting a change plan.
Recommended autonomy	Very High (DELEGATE + REVIEW)
Where Cowork fits	The top-twenty operational attention report is a recurring operational deliverable — a strong Cowork use case.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Extend the production pipeline so every latest grid/hour feature record is
batch-scored after processing.

Requirements:
- read the latest network_feature_table
- produce network_risk_scores with grid_id, timestamp, risk_score,
  risk_level and model_version
- add the scoring task to the existing Airflow DAG, after feature
  generation and before quality_check
- update /network/hotspots and /network/alerts so model scores are
  surfaced through the additive fields designed at API3
- keep existing API contracts backward compatible
- add and update regression tests
- produce a top-20 operational attention report
```

```
First inspect ml/, airflow/, api/ and tests/ and show the cross-component
change plan. Then implement after approval.
```

Learner Validation & Discussion

- Learner reviews the cross-component diff.
- Learner checks that the Airflow scoring task runs only after the features exist.
- Learner validates one scored grid against direct model inference.
- Learner confirms that high risk is still presented as an investigation signal, not a confirmed fault.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`network_risk_scores` has one row per scored grid and interval, with no duplicates.

☐Every score carries a `model_version`.

☐The existing API3 tests pass unchanged after the ML fields are added.

☐The DAG fails cleanly if scoring is attempted before features exist.

☐The top-twenty report names grids, scores and reasons — and uses attention language throughout.

↳ **Connects to:** Phase 7; Capstone

Phase 7 — Claude: AI-Assisted Network Operations

Purpose	Use Claude as the reasoning and engineering-assistance layer above the already-built data, API and ML system. Claude now stops being the tool that helped build the platform and becomes a deliberately engineered part of it.
Storyline	Data and Spark answer “what happened”. ML answers “is this unusual or risky”. Claude answers “what does the evidence mean, what should I inspect next, and how can I safely evolve the software?”
Data scope	**Curated evidence only** — API responses, ML outputs and pipeline status. Never raw rows.
Indicative duration	Approximately 20–22 hours
Autonomy note	Varies by lab. Governance labs (C6, C10, C11) are learner-owned by definition.

C1 — Claude API — Network Insight Generator

■ Data: curated grid evidence object ◆Autonomy: Medium (PAIR)

Business / Training Scenario

The ML system already produces a risk and anomaly score. Claude turns structured evidence into an operations-friendly explanation.

Goal

Introduce the Claude API, model selection and evidence-grounded operational explanation.

Student Activities

1. Compare Claude (chat), the Claude API, Claude Code and Cowork in the context of this project — see the Four Claude Surfaces table in the front matter.
2. Call the Claude API with a structured grid evidence object: current activity, baseline, growth, `peak_ratio`, variability and anomaly score.
3. Ask for a structured response: severity, evidence summary, likely interpretation and investigation steps.
4. Compare model choices for cost, latency and reasoning depth.
5. Experiment with a short versus a richer context package and compare the answers.
6. Require Claude to state explicitly when the evidence is insufficient.

Concepts

- Claude API
- model selection
- structured prompting
- extended thinking
- evidence grounding
- the four Claude surfaces

Expected Output

- network insight response
- model selection rationale

Claude Usage for This Lab

Learner owns	The evidence package and the boundary between observation and interpretation.
Primary AI mode	Claude API — direct integration.
Meaningful AI activity	Learners write the integration. The lesson is what goes into the context, not how to call an SDK.
Recommended autonomy	Medium (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
You are assisting a Network Operations Centre.

Here is the evidence for one grid cell:
  grid_id, timestamp (hourly interval)
  current total_activity, baseline total_activity
  activity_growth, peak_ratio, variability, internet_share
  anomaly_score and direction
  rule alerts currently firing

Respond in exactly four sections:
  SEVERITY        one of NORMAL / ATTENTION / HIGH
  EVIDENCE        only what is stated above, with the numbers
  INTERPRETATION  what this MIGHT mean, clearly marked as inference
  NEXT CHECKS     what a human engineer should inspect next

Rules:
- these are activity measures, not call counts, message counts or MB
- do NOT claim congestion; we have no capacity or utilization data
- if the evidence is insufficient to reach a severity, say so and say
  what additional evidence you would need
- never invent a number that is not in the evidence above
```

Learner Validation & Discussion

- Learner checks that every number in Claude's EVIDENCE section appears in the input — no invented figures.
- Learner verifies that INTERPRETATION is clearly separated from EVIDENCE.
- Learner tests the insufficient-evidence path by removing a field, and confirms Claude says so rather than filling the gap.
- Learner records the model choice and the reason.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The response contains all four sections, every time, across at least five different grids.

☐No response contains the word "congestion" as an assertion.

☐Removing the anomaly score produces an explicit statement of insufficiency, not a confident answer.

☐A model selection rationale is written down with cost and latency considered.

TRAINER FOCUS

Reinforce that Claude is explaining the signal, not inventing a network fault. The response must distinguish observed metrics from suggested investigation — and the four-section format exists to make that distinction impossible to blur.

↳ **Connects to:** C2; C16

C2 — Tool-Using Claude Network Operations Assistant

■ **Data: live API calls** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

A NOC engineer asks natural-language questions. Claude chooses which network API or tool to call before answering.

Goal

Move from prompt-only analysis to an assistant that retrieves live evidence through tools.

Student Activities

1. Expose tools mapping directly onto the endpoints already built: `get_network_summary()` (API1), `get_grid_activity()` (API2), `get_hotspots()` (API3), `get_grid_features()` (API4), `get_anomaly_score()` (ML4/ML6), `get_grid_location()` (API6) and `get_pipeline_status()` (API6). Optionally `get_nearby_hotspots()`.
2. Implement one conversation: “Which areas need attention right now?”
3. Implement the follow-up: “Explain Grid 4821.”
4. Require Claude to call tools rather than rely on stale prompt data.
5. Return evidence references in the final assistant response, naming which tool produced each figure.
6. Add a fallback when a tool or API call fails — the assistant must report the gap, not paper over it.

Concepts

- tool use
- the agent loop
- tool errors
- structured outputs
- why every tool maps to a real endpoint

Expected Output

- working Network Operations Assistant

Claude Usage for This Lab

Learner owns	The tool surface and the definition of what counts as evidence.
Primary AI mode	Claude API with tool use.
Meaningful AI activity	Learners define the tools. Claude Code may implement the wiring.
Recommended autonomy	Medium–High (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
You are the Network Operations Assistant for the Milan grid.
```

```
You have these tools:
  get_network_summary(as_of)
  get_grid_activity(grid_id, as_of)
  get_hotspots(limit, severity, as_of)
  get_grid_features(grid_id)
  get_anomaly_score(grid_id, as_of)
  get_grid_location(grid_id)
  get_pipeline_status()

Rules:
- ALWAYS call a tool for any factual claim. Never answer network questions
  from memory or from earlier in the conversation if the data may have
  changed.
- Before reporting a situation as fact, call get_pipeline_status() and say
  whether the underlying data is currently trustworthy.
- Cite which tool produced each figure.
- If a tool fails, say which one failed and what you therefore cannot
  conclude. Do not substitute an estimate.
- Activity measures are not counts or MB. Do not claim congestion.

First question: "Which areas need attention right now?"
```

FOLLOW-UP PROMPT — SAME SESSION

```
Follow-up: "Explain Grid 4821."

Gather the grid activity, features, anomaly score and location before
answering. Tell me where it is in Milan, what the evidence shows, what it
might mean, and what I should check next. Separate observed evidence from
inference.
```

Learner Validation & Discussion

- Learner confirms Claude actually called the tools rather than answering from context — check the tool-call log.
- Learner deliberately breaks one endpoint and confirms the assistant reports the gap instead of inventing a value.
- Learner checks that every figure in the answer can be traced to a tool response.
- Learner confirms the assistant checked pipeline status before asserting a situation.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Every factual claim in the answer traces to a tool call in the log.

☐With one endpoint disabled, the assistant names the failure and narrows its conclusion.

☐The assistant reports pipeline health as part of a situation answer.

☐No response asserts congestion or converts activity measures into counts or megabytes.

TRAINER FOCUS

This is the core business-facing Claude lab and it directly demonstrates why the APIs built earlier matter — including API6, which exists specifically so that “can I trust this?” has an answer. A team that skipped API6 will discover it here.

Connects to: C3; C12; C14

C3 — Long-Context Incident Investigation

■ Data: curated historical evidence + pipeline status ◆Autonomy: Medium (PAIR)

Business / Training Scenario

An engineer asks whether an abnormal activity pattern has happened before, and whether the data itself can be trusted.

Goal

Teach context engineering using historical network and pipeline evidence.

Student Activities

1. Collect the current grid metrics, recent history, prior alerts, model score and pipeline quality status from DE7 and API6.
2. Compare a “dump everything” prompt against a curated context package.
3. Summarize the older evidence before inserting it into the active context.
4. Ask Claude to separate CURRENT EVIDENCE, HISTORICAL EVIDENCE and UNCERTAINTY.
5. Evaluate whether the answer changes when irrelevant context is removed.
6. Document a context-engineering checklist for the project.

Concepts

- long-context behaviour
- context selection
- summarization
- context engineering
- data trustworthiness as part of the evidence

Expected Output

- incident investigation report
- context engineering checklist

Claude Usage for This Lab

Learner owns	What goes into the context and what stays out.
Primary AI mode	Claude API — context experiments.
Meaningful AI activity	Learners run the comparison. Claude is the subject of the experiment, not the experimenter.
Recommended autonomy	Medium (PAIR)
Where Cowork fits	The incident investigation report is a hand-over document — a Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Investigate an unusual activity pattern at Grid 4821.

I am providing a curated evidence package containing:
- current interval metrics
- summarized recent history (already aggregated, not raw rows)
- prior alerts for this grid
```

```
- the current model risk and anomaly score
- the pipeline status record for the run that produced this data

Answer in exactly three sections:
  CURRENT EVIDENCE      what is true right now, with figures
  HISTORICAL EVIDENCE   whether this has happened before, and how often
  UNCERTAINTY           what you do NOT know, including anything the
                        pipeline status makes doubtful

If the pipeline status indicates rejected rows, handled nulls or a stale
analytics layer, treat that as material and say how it limits the
conclusion.

Do not restate raw rows back to me. Do not claim congestion.
```

Learner Validation & Discussion

- Learner runs the same question with a dump-everything context and with the curated package, and compares the answers side by side.
- Learner checks whether removing irrelevant context changed the conclusion, and records the result.
- Learner confirms the UNCERTAINTY section actually uses the pipeline status rather than producing generic hedging.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Both the dumped and curated runs are recorded, with the answers compared.

☐A written context-engineering checklist exists.

☐When the pipeline status is switched to unhealthy, the UNCERTAINTY section changes materially.

☐The evidence package contains summaries, not raw rows.

TRAINER FOCUS

This lab connects directly to DE8: the pipeline failures learners injected there become part of the incident evidence here. If DE8 was done properly, this lab has real material to work with.

↳ **Connects to:** C4; C16

C4 — Introduce Claude Code to the Existing Repository

■ **Data: the repository** ◆**Autonomy: Medium (PAIR)**

Business / Training Scenario

Instead of a new toy repository, Claude Code is pointed at the Network Intelligence project the learners built themselves.

Goal

Use Claude Code on a real codebase learners already understand.

Student Activities

1. Open the project repository in Claude Code.

2. Ask Claude to explain the repository structure and data flow **without modifying any files**.
3. Locate the Spark pipeline, the Airflow DAG, the API routes, the ML scoring code and the React pages.
4. Create a `CLAUDE.md` stating the architecture, the project rules and the terminology constraints.
5. Include the non-negotiable rules: do not equate high activity with confirmed congestion; the canonical grain is one grid per hourly timestamp; join geography on `properties.cellId`; activity values are not counts or MB.
6. Ask Claude to propose the missing tests and documentation.

Concepts

- Claude Code CLI
- repository understanding
- `CLAUDE.md`
- project context files

Expected Output

- project `CLAUDE.md`
- repository map
- list of proposed missing tests

Claude Usage for This Lab

Learner owns	The contents of `CLAUDE.md` — these are the project's rules, not Claude's.
Primary AI mode	Claude Code — read-only first.
Meaningful AI activity	Claude explains and proposes. The learner writes the rules.
Recommended autonomy	Medium (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Explore this repository WITHOUT modifying any files.

Produce:
1. a repository map: what each top-level directory is responsible for
2. the data flow from landing CSV through to the React dashboard
3. where the Spark pipeline, Airflow DAG, API routes, ML scoring and React
   pages live
4. a list of tests or documentation you believe are missing

Then propose a CLAUDE.md for this project. It must encode at minimum:
- the canonical schema and the raw-versus-analytics grain
- that the analytics grain is one grid per hourly timestamp after
  country-code aggregation
- that activity values are proportional activity measures, not counts or MB
- that high activity must never be described as confirmed congestion
- that geographic joins use properties.cellId, never the 0-based feature id
- that the AS_OF convention defines "now"

Show me the proposed CLAUDE.md. I will edit it before we commit it.
```

Learner Validation & Discussion

- Learner edits the proposed `CLAUDE.md` rather than committing it as generated.
- Learner verifies Claude's repository map against what they actually built.
- Learner tests the rules by asking Claude a question that would violate one, and checking that it does not.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐`CLAUDE.md` exists at the repository root and contains all six rules listed above.

☐The repository map matches reality — spot-check three claims.

☐Asking Claude "is grid 4821 congested?" produces a correction, not an answer.

↳ **Connects to:** C5

C5 — Plan Mode — Add a New NOC Feature Safely

120 min ■ **Data: the repository** ◆**Autonomy: High after approval (DELEGATE)**

Business / Training Scenario

Requirement: add a feature showing the top grids whose activity increased most sharply against baseline.

Goal

Use explore → plan → execute instead of immediately changing a multi-layer application.

Student Activities

1. Use Plan Mode to explore the relevant Spark, SQL, FastAPI and React code.
2. Require a written implementation plan before any edit.
3. Review the proposed files and the API contract change.
4. Approve or revise the plan.
5. Execute the implementation.
6. Run the tests and verify the end-to-end behaviour.

Concepts

- Plan Mode
- impact analysis
- multi-file change planning

Expected Output

- approved plan
- implemented feature
- verification result

Claude Usage for This Lab

Learner owns	The final decision on the plan, and the API contract implications.
Primary AI mode	Claude Code — Plan Mode.
Meaningful AI activity	Claude explores and plans. The learner approves before anything is written.
Recommended autonomy	High after approval (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
New requirement: show the top grids whose activity increased most sharply
against their baseline in the current reporting window.

Use Plan Mode. Do not edit anything yet.

Explore and tell me:
- where this should be computed: Spark, SQL, or the API layer, and why
- which existing baseline logic can be reused rather than duplicated
- what the API contract would look like and whether it can be additive
- which React page it belongs on
- which tests would need to change

Remember the AS_OF convention defines the current reporting window, and
the baseline must exclude the current interval.

Present the plan. I will approve or revise it before you implement.
```

Learner Validation & Discussion

- Learner revises at least one element of the plan before approving it.
- Learner checks that the plan reuses the existing baseline logic rather than adding a third implementation.
- Learner runs the tests after implementation and verifies end to end.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐A written plan existed and was approved before the first edit.

☐The implementation reuses existing baseline logic.

☐The API change is additive and does not break existing clients.

☐All prior tests still pass.

TRAINER FOCUS

The learning point is not “Claude writes code faster”. It is controlled change in a system with multiple dependencies — and the plan is where the learner’s judgement is actually exercised.

↳ **Connects to:** C6

C6 — Permissions & Security Model

■ **Data: the repository** ◆**Autonomy: Learner-owned (REVIEW)**

Business / Training Scenario

Claude Code should be productive without receiving unrestricted authority over data, secrets or destructive commands.

Goal

Configure safe boundaries for agentic coding.

Student Activities

1. Classify operations into allow, ask and deny.
2. Allow safe reads, tests and formatting.
3. Require approval for dependency changes, migrations and configuration edits.
4. Deny raw-data deletion, secrets access and destructive database operations.
5. Demonstrate one blocked operation and one approval-required operation.
6. Discuss managed settings for team environments.

Concepts

- allow / ask / deny
- managed settings
- least privilege
- security boundaries

Expected Output

- project permission policy

Claude Usage for This Lab

Learner owns	The policy. This is a governance decision and is not delegable.
Primary AI mode	Claude Code — configuration.
Meaningful AI activity	Claude may explain the options. The learner decides every classification.
Recommended autonomy	Learner-owned (REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review this repository and propose an allow / ask / deny classification
for Claude Code operations.

Consider specifically:
- reading and writing under data/raw/ (raw must be immutable)
- deleting anything under data/
- editing Airflow DAGs and pipeline configuration
- database migrations and destructive SQL
- dependency changes
- reading .env or any secret
- running the test suite
- formatting and linting

For each, recommend allow, ask or deny AND state the failure you are
preventing. I will make the final classification.
```

Learner Validation & Discussion

- Learner makes the final classification themselves.
- Learner demonstrates one operation that is blocked and one that requires approval.
- Learner confirms raw data deletion is denied.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Deleting anything under `data/raw/` is denied and the denial is demonstrated live.

☐At least one operation is set to ask and is demonstrated prompting.

☐The policy is committed to the repository.

↳ **Connects to:** C7

C7 — Slash Commands & Custom Commands

■ **Data: the repository** ◆**Autonomy: High (DELEGATE)**

Business / Training Scenario

Network engineers and developers repeatedly perform the same checks, so encode them as project commands.

Goal

Create repeatable project workflows as commands.

Student Activities

1. Create commands such as `/network-health`, `/test-api`, `/check-pipeline`, `/review-anomaly` and `/explain-grid`.
2. Define the inputs and the expected output for each command.
3. Run `/test-api` after a code change.
4. Run `/review-anomaly` for a selected grid.
5. Document which commands are engineering-oriented and which are NOC-oriented.

Concepts

- slash commands
- custom commands
- repeatable workflows

Expected Output

- project command set

Claude Usage for This Lab

Learner owns	Which checks are worth encoding.
Primary AI mode	Claude Code — command authoring.
Meaningful AI activity	Claude can write the commands once the learner names the recurring checks.
Recommended autonomy	High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
We repeat these checks constantly. Turn them into project slash commands:

  /check-pipeline    call GET /pipeline/status and summarize health,
                     naming any rejected rows or staleness
  /explain-grid      given a grid_id, gather activity, features, anomaly
```

```
                    score and location, then produce the four-section
                    SEVERITY / EVIDENCE / INTERPRETATION / NEXT CHECKS
                    response
  /review-anomaly   compare the rule alert, classifier output and anomaly
                    score for a grid and explain any disagreement
  /test-api         run the API test suite and summarize failures
  /network-health   run the grain duplicate check on hourly_grid_summary
                    and report pass or fail

For each, define the inputs, the tools it may call, and the expected
output shape. Then implement them.
```

Learner Validation & Discussion

- Learner runs each command at least once and confirms the output shape.
- Learner classifies each command as engineering-oriented or NOC-oriented.
- Learner confirms `/network-health` actually fails when the grain is deliberately broken.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐At least five commands exist and run.

☐`/network-health` detects a deliberately introduced duplicate on (`grid_id`, `timestamp`).

☐Each command has documented inputs and output.

↳ **Connects to:** C8

C8 — Skills — Package Reusable Network Expertise

■ **Data: the repository** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

Instead of re-prompting Claude with the same anomaly interpretation rules every time, package them as skills.

Goal

Turn repeated domain instructions into reusable Claude Skills.

Student Activities

1. Create a `network-anomaly-analysis` skill.
2. Create a `pipeline-troubleshooting` skill.
3. Create an `api-review` skill or a `telecom-data-quality` skill.
4. Define when each skill should activate and what evidence it requires.
5. Compare an answer with and without the relevant skill.
6. Update a skill when a domain rule changes, and observe the effect.

Concepts

- Claude Skills
- reusable expertise
- domain instructions

- when a skill beats a prompt

Expected Output

- 2–4 project skills

Claude Usage for This Lab

Learner owns	The domain rules encoded in each skill.
Primary AI mode	Claude Code — skill authoring.
Meaningful AI activity	Claude drafts; the learner owns the rules, because they are the project's standards.
Recommended autonomy	Medium–High (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Create a network-anomaly-analysis skill for this project.

It should activate when someone asks why a grid is flagged, what an
anomaly score means, or whether a pattern is unusual.

It must encode:
- the required evidence: current activity, baseline, anomaly score and
  direction, rule alerts, and pipeline status
- the four-section response format:
  SEVERITY / EVIDENCE / INTERPRETATION / NEXT CHECKS
- the terminology rules: activity measures not counts or MB; never claim
  congestion; grid is a geographic cell, not a tower
- the instruction to state insufficiency rather than fill gaps

Then show me the same question answered with and without the skill so I
can see what it changed.
```

Learner Validation & Discussion

- Learner runs the same question with and without the skill and documents the difference.
- Learner changes one domain rule and confirms the skill output changes accordingly.
- Learner confirms the skill requires evidence rather than accepting a bare grid ID.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ At least two skills exist.

☐ A documented before-and-after comparison exists for at least one skill.

☐ The anomaly skill refuses to produce a severity when the evidence is incomplete.

↳ **Connects to:** C9

C9 — Subagents & Agent Orchestration

■ **Data: the repository + APIs** ◆**Autonomy: Medium–High (PAIR)**

Business / Training Scenario

A single flagged grid may require data-quality, network-trend, ML and API checks in parallel.

Goal

Use specialist agents for a multi-perspective network investigation.

Student Activities

1. Define specialist subagents: Data Pipeline Agent, Network Analysis Agent, ML Analysis Agent and API Agent.
2. Give each a narrow responsibility and a restricted tool set.
3. Create a parent or supervisor investigation task.
4. Run: “Investigate why Grid 4821 has been flagged.”
5. Collect the specialist findings and produce one combined report.
6. Discuss when multiple agents add value versus adding unnecessary complexity.

Concepts

- subagents
- agent orchestration
- specialization
- aggregation
- knowing when not to orchestrate

Expected Output

- multi-agent investigation report

Claude Usage for This Lab

Learner owns	The division of responsibility between agents.
Primary AI mode	Claude Code — subagents.
Meaningful AI activity	Claude implements; the learner decides the boundaries.
Recommended autonomy	Medium–High (PAIR)
Where Cowork fits	The combined investigation report is a deliverable — assemble it in Cowork.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Investigate why Grid 4821 has been flagged, using four specialist
subagents with narrow responsibilities:

  Data Pipeline Agent   check GET /pipeline/status and the recent run
                        history. Is the underlying data trustworthy?
  Network Analysis Agent  check activity trend and how this grid compares
                        with its own history and with neighbouring cells
  ML Analysis Agent     check the model risk score, the anomaly score and
                        the feature values behind them
  API Agent             verify the endpoints are responding and the data
                        served is fresh

Each agent reports only within its own scope and states its uncertainty.
```

```
Then produce ONE combined report with:
- a single severity
- the evidence from each specialist, attributed
- any disagreement between specialists, called out rather than smoothed
- recommended next checks for a human
```

Learner Validation & Discussion

- Learner checks that each agent stayed within its scope.
- Learner confirms disagreements between specialists are surfaced rather than averaged away.
- Learner argues whether four agents were actually better than one here.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐ Each specialist finding is attributed in the combined report.

☐ At least one disagreement or uncertainty is surfaced.

☐ The learner can state a case where a single agent would have been the better choice.

TRAINER FOCUS

Avoid generic researcher-and-writer demos. This project gives subagents genuinely different responsibilities and genuinely different tools — that is what makes the lab worth running.

↳ **Connects to:** C10

C10 — Hooks & Event-Driven Workflows

■ **Data: the repository** ◆**Autonomy: Medium (PAIR)**

Business / Training Scenario

Automated actions should trigger tests or policy checks at the right moment, not on request.

Goal

Use hooks to enforce engineering discipline around agent actions.

Student Activities

1. Add a post-edit hook that runs the relevant tests after Python changes.
2. Add a pre-action check around sensitive pipeline configuration.
3. Log the hook outcomes.
4. Demonstrate one passing and one failing hook.
5. Discuss which controls belong in hooks and which belong in CI.

Concepts

- hooks
- event-driven workflows
- policy enforcement
- hooks versus CI

Expected Output

- working project hooks

Claude Usage for This Lab

Learner owns	Which controls are worth automating at edit time.
Primary AI mode	Claude Code — hook configuration.
Meaningful AI activity	Claude configures; the learner decides the policy.
Recommended autonomy	Medium (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Add project hooks for this repository.

1. After any edit under spark/ or ml/, run the grain and leakage tests
   automatically — specifically the (grid_id, timestamp) duplicate check
   and the ML2 feature leakage test. These are the two failures that are
   silent and expensive, so they should never wait for CI.
2. Before any edit to airflow/ or pipeline configuration, require an
   explicit confirmation.
3. Log every hook outcome.

Then show me one passing and one failing hook run.
```

Learner Validation & Discussion

- Learner demonstrates a failing hook and confirms it blocks or warns as intended.
- Learner argues which of these belong in hooks and which in CI.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Editing a Spark transformation triggers the grain test automatically.

☐A failing hook is demonstrated live.

☐Hook outcomes are logged.

↳ **Connects to:** C11

C11 — Checkpoints & Safe Rollback

■ **Data: the repository** ◆**Autonomy: Medium (PAIR)**

Business / Training Scenario

Claude Code changes the anomaly threshold logic; the resulting behaviour is worse and needs to be reversed.

Goal

Experiment safely with model or pipeline logic.

Student Activities

1. Create a checkpoint before the change.

2. Modify one risk or anomaly rule.
3. Run the tests and compare the operational output — alert volume, top-twenty composition, agreement with NP3.
4. Identify the regression or the undesirable behaviour.
5. Roll back safely to the checkpoint.
6. Document the decision and what the experiment showed.

Concepts

- checkpoints
- rollback
- safe experimentation

Expected Output

- before-and-after comparison
- successful rollback

Claude Usage for This Lab

Learner owns	The judgement about whether the change was an improvement.
Primary AI mode	Claude Code — checkpointed change.
Meaningful AI activity	Claude makes the change; the learner evaluates and decides.
Recommended autonomy	Medium (PAIR)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Create a checkpoint first.

Then change the anomaly threshold so that roughly twice as many grids are
flagged.

Run the tests and report:
- the change in alert volume
- how the top-20 attention list changed
- whether agreement with the NP3 rule alerts improved or worsened
- any test that now fails

Recommend keep or roll back, and say why. I will decide.
```

Learner Validation & Discussion

- Learner compares operational output, not just test pass or fail.
- Learner performs the rollback and confirms the system returned to the prior behaviour.
- Learner documents why the change was rejected.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐A checkpoint was created before the change.

☐A quantitative before-and-after comparison exists.

☐The rollback is verified — the alert volume returns to the prior value.

↳ **Connects to:** C12

C12 — MCP Server for Network Intelligence

■ **Data: the existing APIs** ◆**Autonomy: High (DELEGATE + REVIEW)**

Business / Training Scenario

The Network Intelligence platform becomes an external capability provider rather than embedding everything in prompts.

Goal

Expose selected network capabilities through MCP so Claude can use them consistently.

Student Activities

1. Design MCP tools and resources for network summary, hotspots, grid metrics, grid location and nearby hotspots, alerts, and pipeline status.
2. Implement or configure a small MCP server **around the existing APIs** — it must call API1–API6, not reimplement them.
3. Connect Claude Code or the assistant to the MCP server.
4. Ask: “Which grids have the highest anomaly scores?”
5. Follow with: “Check whether their data pipeline completed successfully.”
6. Add basic validation and security constraints.

Concepts

- MCP servers
- tool and resource exposure
- integration boundaries
- MCP security

Expected Output

- network intelligence MCP server
- successful tool calls

Claude Usage for This Lab

Learner owns	The integration boundary — what is exposed and what is not.
Primary AI mode	Claude Code — MCP implementation.
Meaningful AI activity	Claude implements the server as a thin wrapper. The learner enforces that it stays thin.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build a small MCP server exposing this platform to Claude.
```

```
Expose these tools, each one a THIN wrapper over the existing endpoint:
  network_summary        -> GET /network/summary
  grid_activity          -> GET /network/grid/{grid_id}
  grid_features          -> GET /network/grid/{grid_id}/features
  grid_location          -> GET /network/grid/{grid_id}/location
  hotspots               -> GET /network/hotspots
  alerts                 -> GET /network/alerts
  pipeline_status        -> GET /pipeline/status

Hard rule: the MCP layer contains NO business logic. It must not compute,
aggregate, threshold or interpret anything. If you find yourself adding a
calculation, the endpoint is missing and we should add it to the API
instead.

Add input validation and basic security constraints.

Then connect and answer: "Which grids have the highest anomaly scores?"
followed by "Check whether their data pipeline completed successfully."
```

Learner Validation & Discussion

- Learner confirms the MCP server contains no business logic — a code review, not a claim.
- Learner verifies each tool returns the same result as calling the endpoint directly.
- Learner checks the second question actually used the pipeline status tool.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐Every MCP tool result matches the corresponding direct API call exactly.

☐No aggregation, threshold or interpretation logic exists in the MCP layer.

☐Both questions are answered using tool calls, verifiable in the log.

TRAINER FOCUS

MCP should expose existing trusted services. Do not allow a second business-logic implementation inside the MCP layer — that is the most common and most damaging mistake in this lab, because it creates two sources of truth that drift apart silently.

↳ **Connects to:** C13; C14

C13 — Plugins for Team Standardization

■ **Data: the repository** ◆**Autonomy: High (DELEGATE)**

Business / Training Scenario

A team should not depend on every developer manually recreating `CLAUDE.md` rules, skills, commands and hooks.

Goal

Package the project conventions so multiple engineers use Claude consistently.

Student Activities

1. Identify the team-standard assets: rules, skills, commands, hooks and approved MCP configuration.

2. Package them into a project or team plugin.
3. Install and use the plugin in a clean project environment.
4. Verify the same engineering safeguards and commands are available.
5. Discuss versioning and ownership.

Concepts

- Claude plugins
- team standardization
- configuration reuse

Expected Output

- Network Engineering Claude plugin

Claude Usage for This Lab

Learner owns	What belongs in the team standard.
Primary AI mode	Claude Code — plugin packaging.
Meaningful AI activity	Claude packages; the learner decides what is standard.
Recommended autonomy	High (DELEGATE)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Package our project conventions into a team plugin containing:
- the CLAUDE.md rules from C4
- the skills from C8
- the slash commands from C7
- the hooks from C10
- the approved MCP server configuration from C12

Then install it into a clean environment and verify that the same
safeguards and commands are available — specifically that /network-health
runs and that the congestion terminology rule is enforced.

Propose a versioning and ownership approach for the plugin.
```

Learner Validation & Discussion

- Learner installs the plugin in a clean environment and verifies the safeguards carry over.
- Learner proposes who owns the plugin and how it is versioned.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

The plugin installs cleanly in a fresh environment.

At least one command and one rule are demonstrably active after install.

A versioning and ownership approach is written down.

↳ **Connects to:** C14

C14 — Claude Agent SDK — Headless NOC Investigation Agent

■ Data: live APIs including `/pipeline/status` ◆Autonomy: High (DELEGATE + REVIEW)

Business / Training Scenario

A service receives “Investigate Grid 4821” and the agent autonomously gathers evidence before returning an investigation brief.

Goal

Move Claude from developer assistant into an application and service workflow.

Student Activities

1. Build an agent using the Claude Agent SDK.
2. Give it the network summary, grid activity, grid location, feature, anomaly and pipeline-status tools built in Phase 4.
3. Implement the investigation workflow: gather → compare → assess → summarize.
4. Return structured fields: severity, evidence, uncertainty and recommended checks.
5. Run it from a headless script or service.
6. Add a failure path for when one evidence source is unavailable.

Concepts

- Claude Agent SDK
- headless agents
- tool orchestration
- structured output
- graceful degradation

Expected Output

- headless NOC investigation agent

Claude Usage for This Lab

Learner owns	The investigation workflow and the degradation behaviour.
Primary AI mode	Claude Agent SDK.
Meaningful AI activity	Claude Code implements the agent; the learner defines the workflow and the failure path.
Recommended autonomy	High (DELEGATE + REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Build a headless investigation agent using the Claude Agent SDK.

Input: a grid_id. Output: a structured investigation brief.

Tools available:
  network_summary, grid_activity, grid_features, grid_location,
  anomaly_score, pipeline_status

Workflow: gather -> compare -> assess -> summarize.
```

```
Always call pipeline_status FIRST. If the pipeline is unhealthy, that
must appear in the uncertainty field and must constrain the severity.

Return structured fields:
  severity            NORMAL / ATTENTION / HIGH
  evidence            list of {claim, value, source_tool}
  uncertainty         what is unknown or untrustworthy
  recommended_checks  what a human should inspect next

Failure path: if any tool is unavailable, complete the investigation with
the remaining evidence, name the missing source in uncertainty, and lower
confidence accordingly. Never substitute an estimate for a failed call.

Run it headlessly for grid 4821.
```

Learner Validation & Discussion

- Learner runs the agent headlessly and inspects the structured output.
- Learner disables one tool and confirms the agent degrades gracefully rather than failing or fabricating.
- Learner confirms every evidence item names its source tool.
- Learner confirms pipeline status was called first.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The agent runs from a script with no interactive input.

☐Every evidence item carries a `source_tool`.

☐With one tool disabled, the agent still returns a brief and names the missing source.

☐With the pipeline unhealthy, the severity assessment is visibly constrained.

↳ **Connects to:** C15; Capstone

C15 — Headless / CI Engineering Review

■ **Data: the repository + a change diff** ◆**Autonomy: Advisory only (REVIEW)**

Business / Training Scenario

After a code change, CI can ask Claude to review the likely impact without giving it deployment authority.

Goal

Use Claude as a controlled reviewer in engineering workflows.

Student Activities

1. Run the standard tests first — Claude reviews in addition to tests, never instead of them.
2. Provide Claude with the relevant change context and the project rules from `CLAUDE.md`.
3. Ask it to identify potential data-quality, API-contract, security or missing-test risks.
4. Produce a review report.
5. Keep deployment and manual approval outside the agent.
6. Compare the findings against the normal static and test results.

Concepts

- CI use
- review automation
- human approval boundaries

Expected Output

- AI-assisted engineering review

Claude Usage for This Lab

Learner owns	What the review is allowed to gate, and what stays with a human.
Primary AI mode	Claude — headless CI review.
Meaningful AI activity	Claude reviews and reports. It never approves or deploys.
Recommended autonomy	Advisory only (REVIEW)

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
Review this change against the project rules in CLAUDE.md.

The test suite has already run and its results are attached.

Identify risks in these categories specifically:
- data-grain: could this reintroduce duplicates on (grid_id, timestamp)?
- leakage: could this let a feature see data after feature_timestamp?
- geographic join: does this touch the cellId join anywhere?
- API contract: is any change to a response shape non-additive?
- terminology: does any new string assert congestion or convert activity
  measures into counts or MB?
- missing tests

Produce a review report. Do NOT approve, merge or deploy anything —
those decisions stay with a human.
```

Learner Validation & Discussion

- Learner compares Claude's findings against the static analysis and test results, and notes what each caught that the other did not.
- Learner confirms the agent has no deployment authority.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐The review report covers all six named risk categories.

☐At least one finding is something the test suite did not catch — or the learner explains why not.

☐No approval or deployment action is available to the agent.

↳ **Connects to:** C16

C16 — Context, Cost & Usage Optimization

■ Data: comparison lab ◆Autonomy: Medium (PAIR)

Business / Training Scenario

Compare an inefficient design that sends raw records to Claude against an efficient tool-driven design that sends only relevant evidence.

Goal

Teach efficient LLM architecture using the project's real data scale.

Student Activities

1. Estimate the context size of a raw multi-grid extract — use the real row counts from the accumulated history.
2. Create a curated ten-to-twenty record evidence package.
3. Compare answer quality, latency and cost implications between the two.
4. Use model selection based on task difficulty.
5. Apply summarization and context compaction for long sessions.
6. Define the project rules for when to use cheaper and faster versus deeper reasoning models.

Concepts

- context engineering
- cost management
- model selection
- long-session management

Expected Output

- cost and context design guideline

Claude Usage for This Lab

Learner owns	The project rules for model and context selection.
Primary AI mode	Claude — comparison experiments.
Meaningful AI activity	Claude is the subject of the measurement.
Recommended autonomy	Medium (PAIR)
Where Cowork fits	The cost and context design guideline is a standing reference document — a Cowork deliverable.

SUGGESTED LEARNER PROMPT — PASTE INTO CLAUDE / CLAUDE CODE

```
We are comparing two designs for the same question:
"Which grids need operational attention right now, and why?"

Design A: send the raw hourly rows for all grids in the current window
          directly in the prompt.
Design B: call the hotspot and anomaly endpoints and send only the top 20
          curated evidence records.

For each design, estimate the context size using our real row counts,
```

```
and tell me:
- how the answer quality differs
- how latency and cost differ
- what Design A gets wrong that Design B gets right, and vice versa

Then propose project rules for when to use a cheaper/faster model versus
a deeper reasoning model in this system.
```

Learner Validation & Discussion

- Learner computes the actual context size from real row counts rather than accepting an estimate.
- Learner runs both designs and compares the answers.
- Learner writes the project model-selection rules themselves.

ACCEPTANCE CRITERIA — LAB IS NOT COMPLETE UNTIL ALL PASS

☐A real context-size figure is computed from the project's own data.

☐Both designs are run and compared on quality, not just cost.

☐A written model-selection guideline exists and names specific tasks.

TRAINER FOCUS

This lab should land the principle the whole phase rests on: **LLMs reason over the right evidence; Spark and SQL remain responsible for large-scale computation.** If a team is still tempted to send raw rows after this lab, revisit the non-negotiable architecture principle.

↳ **Connects to:** Capstone

Final Integrated Capstone — AI-Powered Network Operations Control Room

CAPSTONE SCENARIO

In the latest hourly reporting interval, several geographic grids show unusual mobile activity. The operations team must identify the most important areas, determine whether the behaviour is a recurring peak or an unusual deviation, verify that the data pipeline is healthy, inspect the spatial pattern on the Milan grid map, and produce a prioritized investigation brief using the completed platform.

End-to-end flow

Raw Network Activity

↓

Airflow ingestion & validation

↓

Validated Raw Zone

↓

PySpark processing & geographic enrichment

↓

Processed Parquet + Analytics Warehouse

↓

Feature generation

↓

ML Risk / Anomaly Scoring

↓

FastAPI (including /pipeline/status)

↓

React NOC Dashboard + Claude Tools / MCP

↓

Claude Network Operations Assistant

↓

Prioritized Investigation Brief

Required demonstration

7. Trigger or run the ingestion pipeline and show successful processing of a file that has not been processed before.
8. Show the processed and warehouse data, and the current network activity KPIs. State the `AS_OF` value being used.
9. Show the high-activity and anomaly scores for the current reporting window.
10. Open the React NOC dashboard, identify the top operational-attention grids, and show their geographic pattern on the Milan grid map.
11. Verify on the map that the highlighted polygon for the top hotspot matches the centroid returned by `/network/grid/{id}/location`.
12. Ask Claude: *"Give me the network situation for the last hour."*
13. Ask Claude: *"Investigate the highest-risk grid."*
14. Claude must call tools or services to retrieve the evidence rather than fabricate values — show the tool-call log.
15. Ask Claude to state its uncertainty and its recommended next checks.

16. Demonstrate one pipeline or data-quality failure scenario, and show how the assistant reports it through `/pipeline/status` rather than continuing as if the data were sound.
17. Present the architecture and explain the responsibility boundaries between Spark, ML and Claude.

> Item 10 is the one that separates a working demo from a trustworthy system. Any team can show a green pipeline. The question worth grading is what the assistant says when the pipeline is **not** green — and whether it noticed without being told.

Trainer evaluation rubric

Area	Weight	What good looks like
Data & Spark processing		Correct schema, correct grain with the duplicate assertion in code, correct geographic join validated geographically, reproducible outputs.
Data Engineering		Clear zones, orchestration, quality controls, a machine-readable pipeline status record, and demonstrated safe reruns.
API & Dashboard		Stable service contracts, a shared `AS_OF`, usable operational views, and a map that highlights the correct cells.
ML		Appropriate problem framing with a future-window label, simple explainable features, chronological validation, base rate reported, no overclaiming.
Claude Assistant		Tool-grounded investigation, evidence and uncertainty separated, pipeline health checked before asserting a situation, safe agent behaviour.
Architecture & Communication		The team can explain why each component exists and how data flows, without notes.

> **⚠ GRADE THESE TWO THINGS EXPLICITLY**
>
> Two automatic deductions worth stating to learners in advance, because both look like success:
>
> **A model reporting near-perfect accuracy** without an investigation into why. On this data that is a symptom, not an achievement.
>
> **A map that renders beautifully but is joined on the wrong identifier.** Ask every team which key they joined on and have them prove it with a centroid.

Trainer Transition Scripts

Say these out loud between phases. They are what keeps forty labs feeling like one system rather than a technology tour.

Transition	Script
Python → PySpark	We know how to process and reason about one daily file locally. Now apply the same rules across many daily files and millions of grid, hour and country-code activity records. The business logic stays familiar; the execution model changes.
PySpark → Data Engineering	You have a Spark job that works. Production engineering begins when that job must run reliably every day, reject bad data, publish trusted outputs, and tell us when something failed.
Data Engineering → API / React	Processed data has value only when operational consumers can retrieve it safely. We now turn warehouse outputs into stable services and a lightweight NOC view — including a service that answers whether the data can be trusted right now.
Dashboard → ML	The dashboard tells us what happened. The next question is whether current behaviour looks unusual enough to deserve attention — and whether we can say anything about the next hour rather than only about this one.
ML → Claude	The model produces a signal, not an investigation. Claude will not replace the model. It will collect evidence, explain the signal, compare context and suggest what an engineer should inspect next.
Claude API → Claude Code	We have used Claude inside the application. Now we use Claude as an engineering agent on the same codebase — under rules, permissions and checkpoints.
Claude Code → Agent SDK / MCP	Claude can help build the platform. Next we make selected platform capabilities available to an agent in a controlled, reusable way.

Claude Topic Coverage Map

Claude topic	Where covered
The four Claude surfaces: chat, Code, Cowork, API	Front matter; C1; C4
Claude API deep-dive & model selection	C1; C16
Extended thinking & long-context behaviour	C1; C3
`CLAUDE.md` & project context files	C4
Claude Code CLI — agentic coding	C4–C11
Plan Mode	C5
Permissions & security model	C6
Slash commands & custom commands	C7
Skills	C8
Subagents & agent orchestration	C9
Hooks & event-driven workflows	C10
Checkpoints & safe rollback	C11

Claude topic	Where covered
MCP servers inside Claude Code	C12
Plugins for team standardization	C13
Claude Agent SDK & headless / CI use	C14; C15
Context engineering for long sessions	C3; C16
Cost & usage management	C16
Claude as a build partner across the whole stack	NP1–ML6, in every lab's Claude Usage section
Cowork for written deliverables	NP1; SP5; DE1; DE4; DE7; DE8; ML1; ML3; ML6; C3; C9; C16
Building a project end-to-end	C1–C16 + Final Capstone

Repository Structure

One canonical tree. Directories marked *(Phase 7)* are created empty early and populated when their syllabus section begins.

```
network-intelligence/
│
├── data/
│   ├── landing/                  incoming daily CSVs
│   ├── raw/                      immutable accepted CSVs
│   ├── rejected/                 quarantined files + reasons
│   ├── reference/
│   │   └── milano-grid.geojson   static, never date-partitioned
│   ├── processed/                Parquet, partitioned by date
│   └── analytics/                curated Parquet / warehouse outputs
│
├── logs/                         ingestion audit, run history, pipeline status
│
├── ingestion/
│   ├── file_detector.py
│   ├── schema_validator.py
│   └── ingestion_logger.py
│
├── spark/
│   ├── schemas.py
│   ├── cleaning.py
│   ├── aggregations.py
│   ├── enrichment.py
│   └── telecom_pipeline.py
│
├── airflow/
│   └── network_pipeline_dag.py
│
├── api/
│   ├── main.py
│   ├── routers/                  network, features, prediction, pipeline
│   ├── services/
│   └── models/
│
├── ml/
│   ├── features.py
│   ├── train.py
│   ├── anomaly.py
│   ├── predict.py
│   └── batch_score.py
│
├── frontend/
│   ├── public/
│   │   └── reference/            read-only static copy of the GeoJSON for maps
│   └── src/
│
├── agents/                       (Phase 7) Agent SDK NOC investigation agent
│   └── noc_investigator.py
│
├── mcp/                          (Phase 7) Network Intelligence MCP server
│   └── network_mcp_server.py
│
├── tests/
│   ├── test_ingestion.py
│   ├── test_spark.py
│   ├── test_api.py
│   └── test_ml.py
│
├── docs/
│   ├── architecture.md
│   ├── data_contract.md
│   ├── api_contracts.md
│   └── runbook.md
│
├── .claude/                      (Phase 7) commands, skills, subagents, hooks
```

```
├── CLAUDE.md
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

Directory responsibilities

Directory	Responsibility
`data/` + `ingestion/`	Landing, raw, rejected, reference, processed and analytics layout; daily CSV validation and routing. `milano-grid.geojson` stays static under `data/reference/`.
`logs/`	Ingestion audit records, Airflow run history, and the machine-readable pipeline status record that API6 serves.
`spark/`	Canonicalization, null handling, country-code-to-grid/hour aggregation, GeoJSON enrichment, Parquet outputs, `telecom_pipeline.py`.
`airflow/`	DAGs for ingest → Spark → warehouse → quality check → scoring → notify.
`api/`	FastAPI endpoints: `/network/summary`, `/network/grid/{grid_id}`, `/network/grid/{grid_id}/features`, `/network/grid/{grid_id}/location`, `/network/hotspots`, `/network/alerts`, `/network/predict-risk`, `/pipeline/status`.
`ml/`	Feature engineering, the simple classifier and anomaly logic, batch scoring.
`frontend/`	Lightweight React NOC dashboard including the Milan geographic hotspot view.
`tests/`	Validation, API, pipeline and ML tests; later also invoked by Claude Code hooks and CI.
`docs/`	Architecture, the data-grain contract, API contracts, runbooks and trainer notes. `CLAUDE.md` sits at the repository root.

Appendix A — Lab 0: Environment Setup

Run this **before** the first teaching session, not during it. An unprepared room loses the whole of Phase 1 to environment problems, and none of that time teaches anything about telecom data.

Components and versions

Component	Guidance	Needed from
Python	3.10 or 3.11. Avoid the newest release — PySpark and Airflow wheels lag behind.	NP1
Java (JDK)	JDK 11 or 17. Required by Spark. Set `JAVA_HOME`.	SP1
PySpark	Match the version to the JDK. Local mode is sufficient throughout.	SP1
Airflow	Standalone or Docker. **On Windows, use WSL2 or Docker** — native Windows Airflow is not supported and will consume a session.	DE2
Database	PostgreSQL preferred. SQLite is acceptable only for a small day count	DE6
Node.js	LTS release, for the React dashboard.	RE1
Map library	react-leaflet or MapLibre. Decide before RE4 and install it at setup.	RE4
Claude Code	Installed and authenticated for every learner.	NP1
Claude API access	API key provisioned and tested. Do not discover a billing problem at C1.	C1

Setup smoke test

Every learner runs these five checks and reports green before session one. Any red is resolved offline.

18. `python --version` returns 3.10 or 3.11.
19. `java -version` returns 11 or 17, and `JAVA_HOME` is set.
20. A PySpark session starts and `spark.range(10).count()` returns 10.
21. The database accepts a connection and a `CREATE TABLE` / `DROP TABLE` round trip.
22. Claude Code opens in the project folder and can list the files.

Files supplied to learners

File	Placed at	Note
`sms-call-internet-mi-YYYY-MM-DD.csv` (one per day)	`data/landing/` — with two or three days held back by the trainer	The held-back files are used at DE2, DE7 and DE8 to simulate arrivals and reruns.
`milano-grid.geojson`	`data/reference/`	Static. Never enters the landing zone.

> Supply these files directly. Do not have learners download the full public archive: it contains a second city and several reference sets whose identifiers collide with Milan, and the resulting corruption is silent. Keep the filenames exactly as above — they appear throughout this guide, in the ingestion glob, and in the Spark reader.

Appendix B — Known Traps, Consolidated

Every one of these produces a **wrong answer with no error message**. That is what makes them worth a page of their own. Brief the room on the first four before Phase 1 begins; each is also repeated in the lab where it bites.

#	Trap	Where it bites	How it looks when it happens	The check that catches it
1	Joining geography on the top-level GeoJSON `id` (0-based) instead of `properties.cellId` (1-based)	SP4, RE4	Join succeeds, coverage reports 100%, map renders — and every hotspot is on its neighbour's polygon	Geographic spot-check: compare a cell's centroid against `/network/grid/{id}/location`. A coverage percentage cannot detect this.
2	Missing or partial country-code aggregation	SP3 onward	Every KPI, feature, model score and dashboard number is inflated proportionally, so nothing looks obviously wrong	Assert zero duplicates on (`grid_id`, `timestamp`) in code, and confirm the summary row count is strictly below the cleaned row count.
3	Circular ML label — features and label computed over the same window	ML1, ML2, ML3	Accuracy near 99%. The team believes the model works and the evaluation discussion becomes meaningless	Features from the trailing window ending at `t`; label describes `t+1`. Report the base rate. Investigate any accuracy above ~95%.
4	A per-grid, per-hour-of-day baseline built from a single day	NP3	Baseline equals the current value, every deviation is zero, the alert file is empty and learners assume a bug	Use the within-day baseline at NP3. The hour-of-day baseline arrives at ML4, once history exists.
5	Globbing `sms-call-internet-*.csv` instead of `sms-call-internet-mi-*.csv`	SP1, DE2	Row count roughly doubles, enrichment coverage silently halves	Put the `-mi-` in the glob. Assert every `grid_id` falls within 1–10000.
6	Each layer inventing its own definition of "now"	API1, RE2, C2, Capstone	The dashboard, the API and the assistant quietly disagree about the current window	One configured `AS_OF`, returned in every response. Verify API1 and `/pipeline/status` report the same value.
7	Feature names drifting between `ml/features.py` and API4	API4, ML5, RE5	The prediction endpoint fails or, worse, silently receives a default value	Compare the two field lists character for character. One test asserting the names match.
8	The MCP layer growing its own business logic	C12	Two sources of truth that drift apart with no visible failure	Code-review the MCP server for any calculation. Assert each tool result matches the direct API call.
9	The assistant narrating pipeline health from memory	C2, C3, C14, Capstone	Confident answers about data that is stale or partially rejected	Require `get_pipeline_status()` before any situation assertion. Disable an endpoint and confirm the assistant reports the gap.
10	Rendering all 10,000 polygons as SVG	RE4	The page becomes unresponsive and learners debug their React instead of their approach	Choose a rendering strategy before implementing. Fetch the GeoJSON once.

Checks that are supposed to find nothing

State these before learners run them, otherwise the room debugs working code:

- **Negative activity values** — none exist in genuine files. Activity values are non-negative proportional measures.
- **Exact duplicate rows** — none exist in genuine files. The raw grain is naturally unique.
- Both checks earn their place at DE2 and DE8, where faulty files are deliberately injected and they fire as designed.

Appendix C — Indicative Programme Timing

Lab durations are hands-on time and exclude the trainer's introduction and the closing discussion. Add roughly 20% for a room new to the domain.

Phase	Labs	Hands-on hours	Suggested sessions
Phase 1 — Core Python	NP1–NP3		
Phase 2 — PySpark	SP1–SP7		
Phase 3 — Data Engineering	DE1–DE8		
Phase 4 — FastAPI	API1–API6		
Phase 5 — React	RE1–RE5		
Phase 6 — Machine Learning	ML1–ML6		
Phase 7 — Claude	C1–C16		
Capstone	—		
Total	**51 labs**		

> If the programme must be compressed, cut depth rather than coverage in Phase 5 (React is an API consumer, not a frontend course) and in C7, C10, C11 and C13, which can be demonstrated rather than built by each learner. Do **not** compress SP3, SP4, ML1 or API6 — those four are where the silent failures live.

Final Trainer Principles

- Keep the same system alive across modules. Reuse artifacts, APIs, schemas and outputs.
- Do not introduce customer-retention or churn concepts into this network activity track.
- Treat `grid_id` as a geographic activity cell unless a physical network mapping is explicitly supplied. The core downstream analytics grain is one grid per hourly timestamp after country-code aggregation.
- Use “high activity”, “hotspot”, “unusual activity” and “operational attention” precisely. Treat the SMS, call and internet values as activity measures rather than literal counts or megabytes, and do not claim confirmed congestion from activity data alone.
- Keep ML simple, interpretable and operational — and make sure it is predicting something rather than restating it.
- Make Claude use tools and curated evidence. Do not ask the model to perform the role of Spark or SQL.
- Require the assistant to distinguish observed evidence, model output, inference and recommended next checks.
- Claude Code exercises operate on the same repository learners built, so agentic coding is grounded in a familiar architecture.
- Use permissions, hooks, checkpoints and CI reviews to teach that agentic engineering must be governed, not merely powerful.
- The final learner story should be: **Observe → Process → Engineer → Predict → Explain → Act.**

> The single sentence to leave the room with: *the hard part was never getting Claude to write the code — it was knowing what to check.*